

Efficient Deep Learning

CPSC680: Trustworthy Deep Learning

Rex Ying

Readings

- Readings are updated on the website (syllabus page)
- **Readings:**
 - [Data Distillation](#)
 - [Sparse Training](#)
 - [Sparse Attention](#)
 - [Deep Learning Efficiency Survey](#)

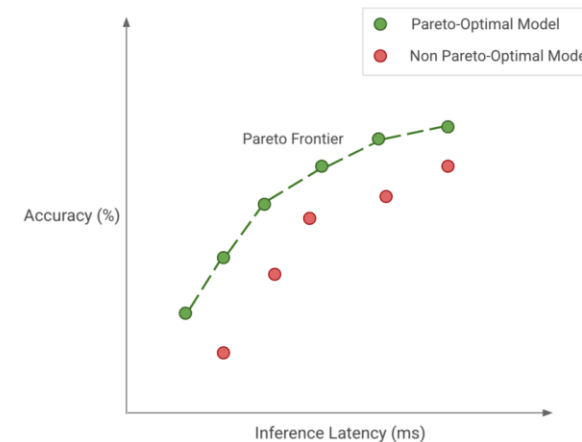
Environmental Well-Being of Deep Learning

- Alignment: deep learning should conform to **the fundamental values of the society** in which they are deployed
 - Large-scale graph datasets → demands training efficiency
 - Deeper or more complex architectures → requires more efficient models
 - Growing size of parameters and Flops → challenge for deployment on edge devices



Compute Clusters

Credit: Imaginima/E+/gettyimages



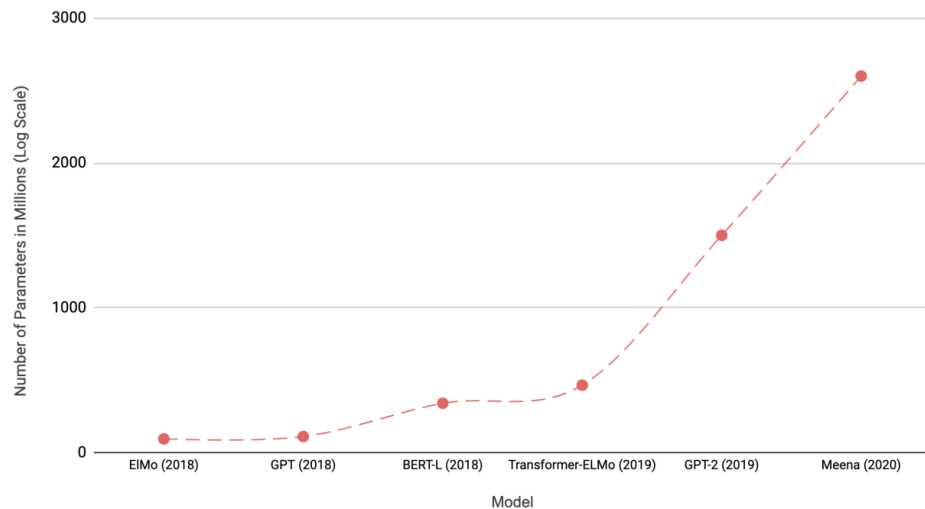
Performance vs. Efficiency

Credit: [Efficient Deep Learning: A Survey on Making Deep Learning Models Smaller, Faster, and Better \(arxiv.org\)](#)

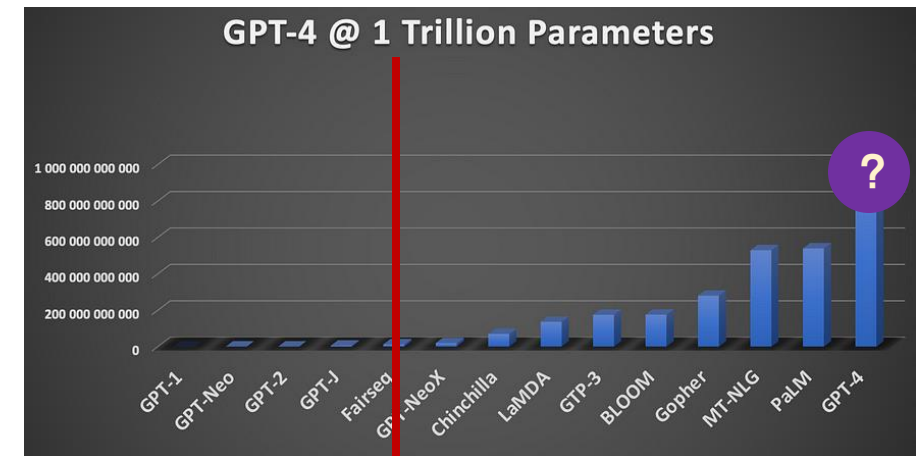
Modern Models are Expensive

- Parameter scaling has been exponential in both NLP and CV communities

Before 2020: single digit billion parameters



Since 2022: 100+ billion parameters



2022

Efficient Deep Learning Strategies

- **Compression**

- Methods to reduce computation cost by reducing operations required per **training / fine-tuning iteration** and **inference**
- Quantization, Pruning, sparse computation

- **Architecture**

- Distillation, efficient minibatching (graphs)

Compression

**Learning
Techniques**

Automation

Infrastructure

Efficient Deep Learning Strategies

- **Automation**

- Sample-efficient AutoML
- Hyperparameter search
- Potentially very powerful in democratizing AI

- **Infrastructure**

- Efficient CUDA kernels
- Edge device computing: TFLite, PyTorch Mobile

Compression

**Learning
Techniques**

Automation

Infrastructure

Content

- Distillation
 - Model distillation
 - Data distillation
- Quantization
- Sparse Neural Networks
- Sparse Attention

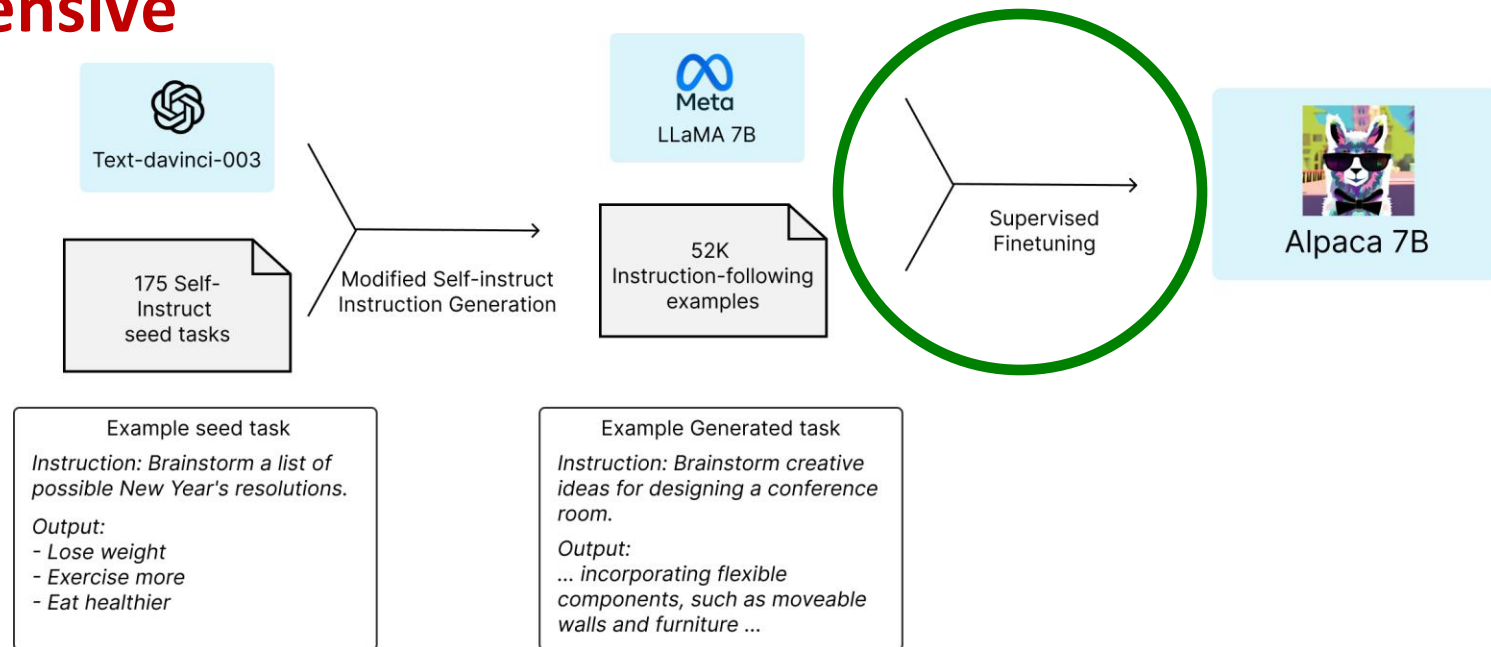
Content

- Distillation
 - Fine-tuning
 - Model distillation
 - Data distillation
- Quantization
- Sparse Neural Networks
- Sparse Attention

Learning Techniques

Alpaca vs. LLaMa

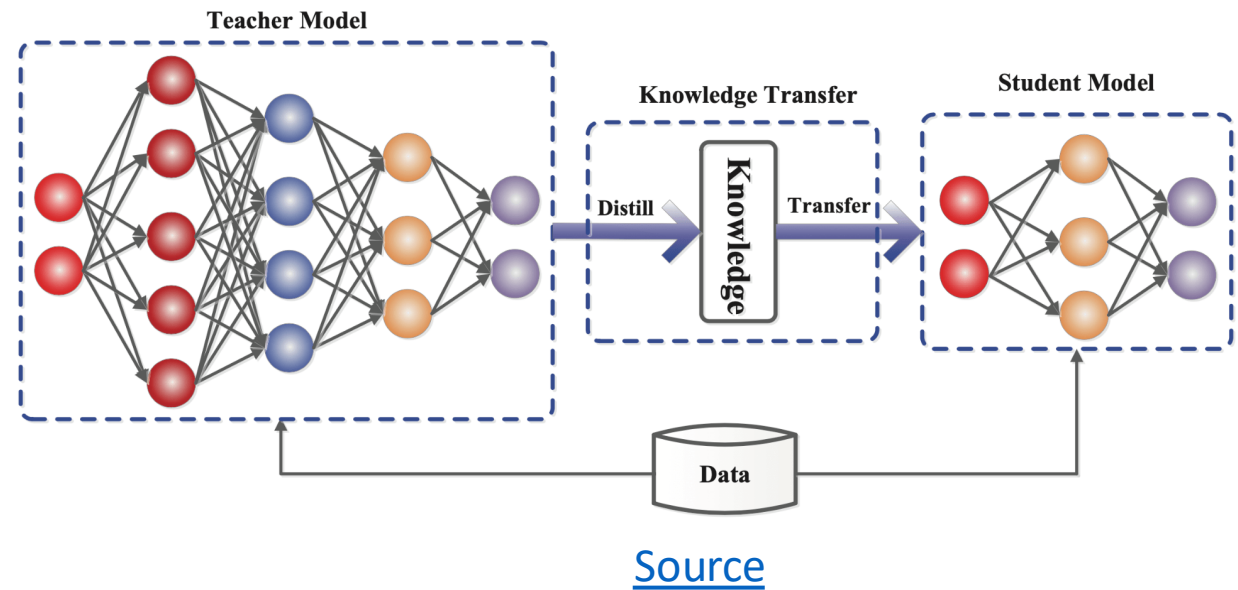
- Instead of distillation, fine-tuning is also a popular alternative
- Embeddings of the pre-trained models are fixed
- Training becomes **a lot more efficient**
- Inference can still be **expensive**



Models Distillation

- **Model distillation**

- Learn **student model(s)** that are light-weight but can be more efficient
- Simple models also tend to be more **explainable**



Model Distillation

- Smaller networks (students) can be taught to extract 'dark knowledge' from larger models / ensembles of larger models (teachers)

Distillation Loss

- In the original [formulation](#), the student minimizes a weighted combination of two objectives:

- $\mathcal{L}_{NLL}(z_s, y) = -\sum_{j=1}^c \boxed{y_j} \log \sigma_j(\boxed{z_s})$

Performance

Groundtruth

Student model prediction

- $\mathcal{L}_{KD}(z_s, z_t) = -\tau^2 \sum_{j=1}^c \sigma_j\left(\frac{\boxed{z_t}}{\tau}\right) \log \sigma_j\left(\frac{\boxed{z_s}}{\tau}\right)$

Fidelity

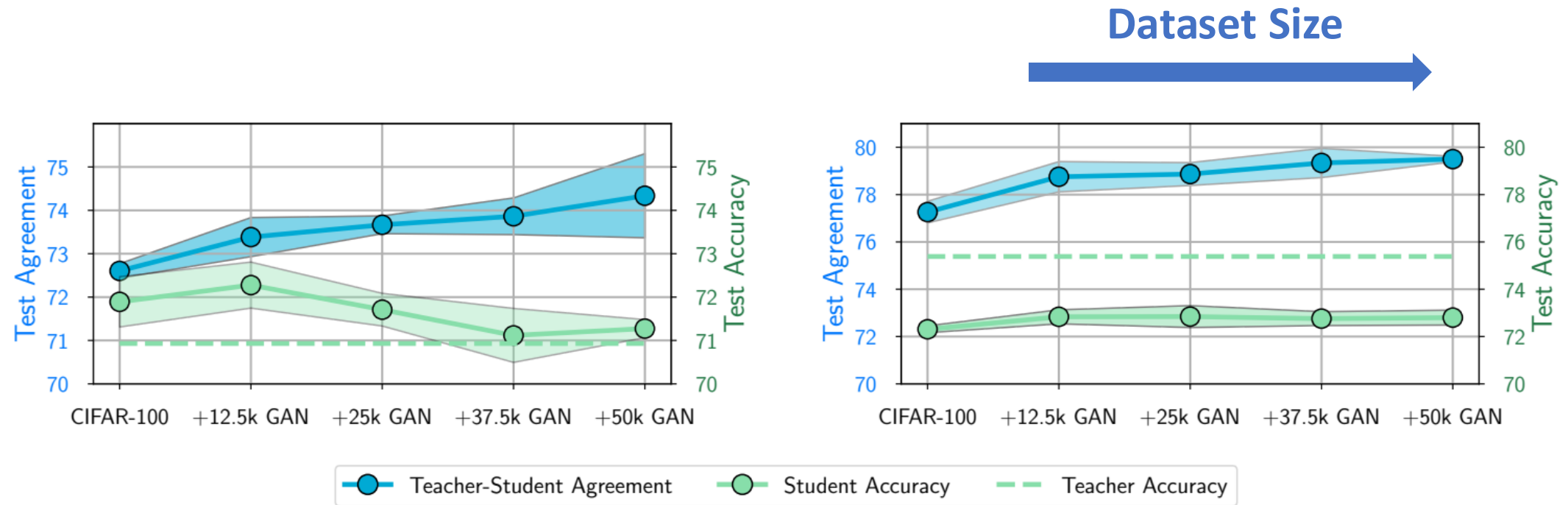
Teacher model prediction

What happens when the temperature τ is high / low?

- Final loss is a convex combination of the two components

Student Model Performance

- Agreement: how much do student predictions agree with teacher predictions



(a) Self-distillation

(b) Ensemble distillation

[Does Knowledge Distillation Really Work?](#),
NeurIPS 2021

Data Condensation

- **Data distillation**

- Find subset of training data that are representative
- Such that model trained on the subset can achieve similarly good performance

- Example: [coreset selection](#)

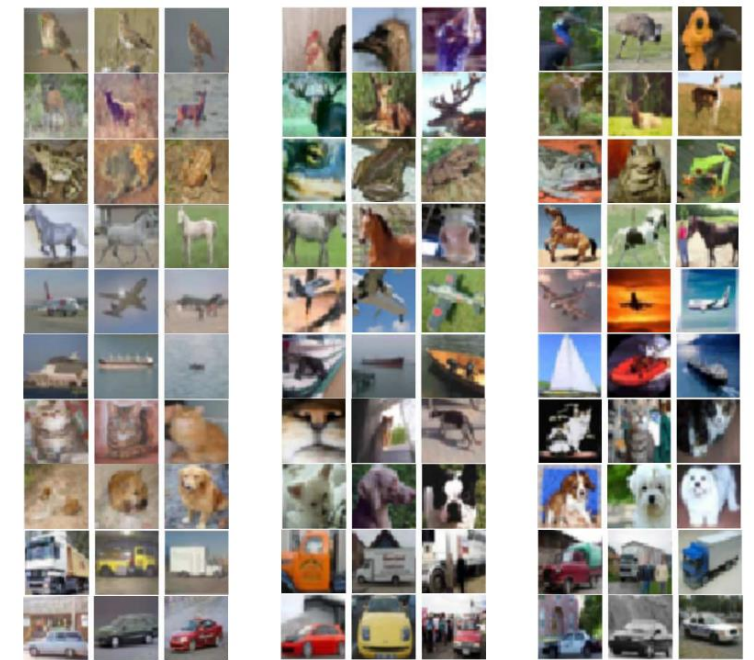
- At every epoch, a subset of **representative** images are selected for training. **Objective:**

Training Subset $S^* = \arg \min_{S \subseteq V, \gamma_j \geq 0 \forall j} |S|, \text{ s.t.}$

$$\max_{w \in \mathcal{W}} \left\| \sum_{i \in V} \nabla f_i(w) - \sum_{j \in S} \gamma_j \nabla f_j(w) \right\| \leq \epsilon.$$

Model weights

Gradient difference (between subset and the whole training set)



(a) First

(b) Middle

(c) Last

Epoch 1

Epoch 100

Epoch 200



Two-stage Optimization

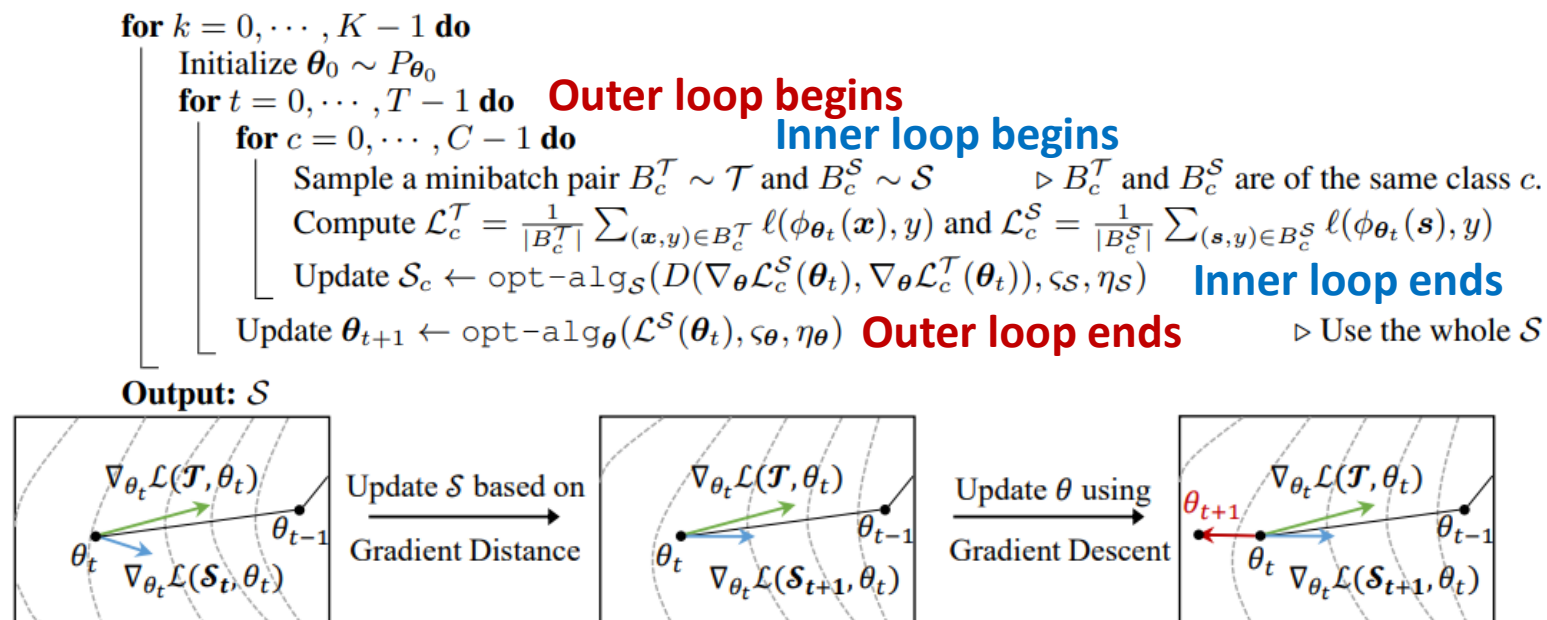
- “**Inner loop**” that trains a model on learned data
 - $\theta(S) = \arg \min_{\theta} \mathcal{L}(S, \theta)$
- “**outer loop**” that optimizes the learned data for performance on natural (i.e., unmodified) data
 - $\min_S \mathcal{L}(T, \theta(S))$ such that $\theta(S) = \arg \min_{\theta} \mathcal{L}(S, \theta)$

Original Dataset **Trained parameters on the subset S**

- **Gradient Matching**: directly encourages the training dynamics on the synthetic set to mimic that on the real training set

Gradient Matching

- **Gradient Matching**: directly encourages the training dynamics on the synthetic set to mimic that on the real training set
- At any time step t , for every class c , create samples S
 - S is usually the output of a generative model that “**summarizes**” the training set



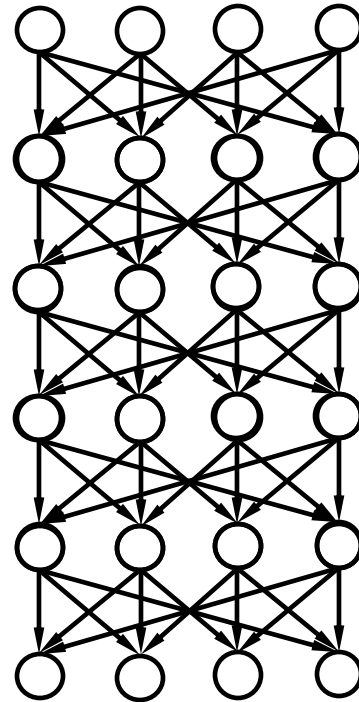
Content

- Distillation
 - Model distillation
 - Data distillation
- Quantization
- Sparse Neural Networks
- Sparse Attention

Compression

Sparse Structure of Neural Networks

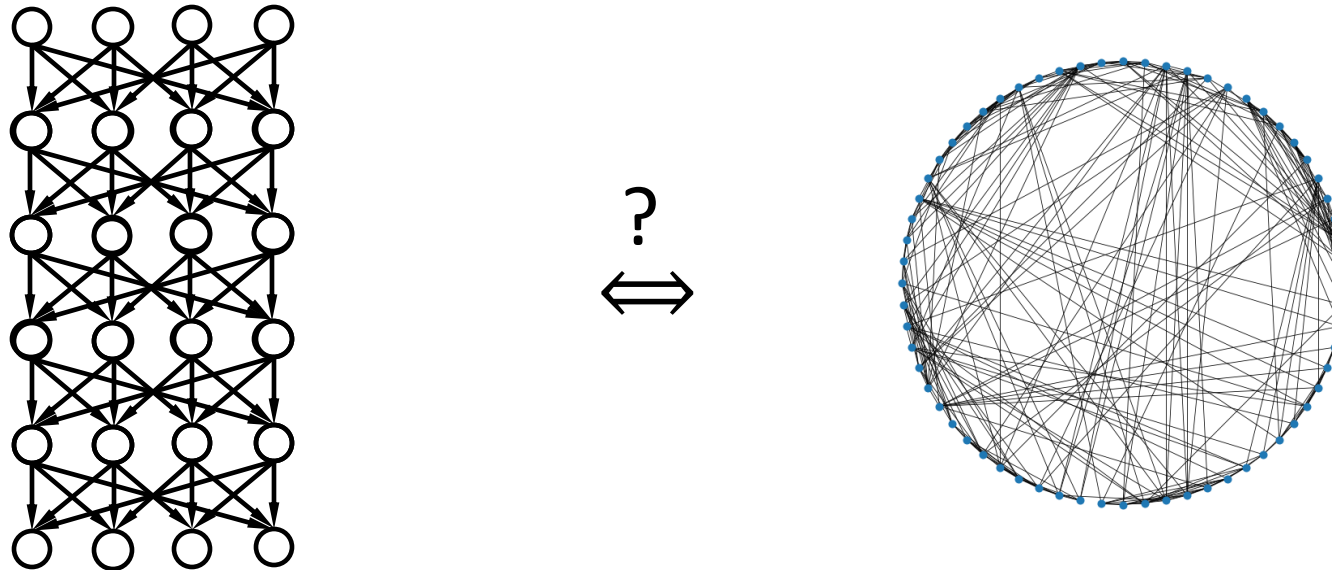
- **Neural networks consist of neurons**
- We know connections between neurons affect NN performance
- *But how?*



Sparse Structure of Neural Networks

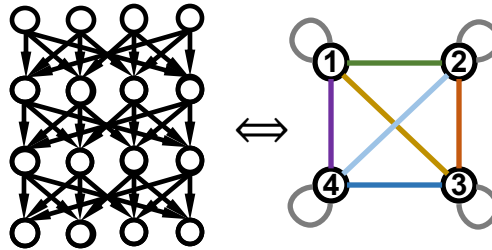
- **Underneath a NN, there is a graph**

- We want to find a proper **graph representation of NN** to answer:
- *Is there a link between the **graph structure** and **NN performance**?*
- *If so, what are **structural signatures** of well-performing NNs?*
- *Can these signatures **generalize** across tasks and datasets?*

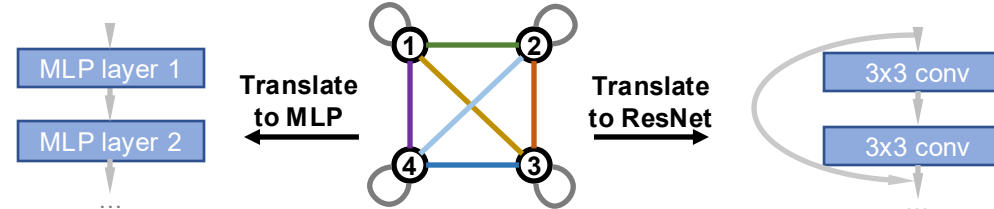


Graph Structure of Neural Networks

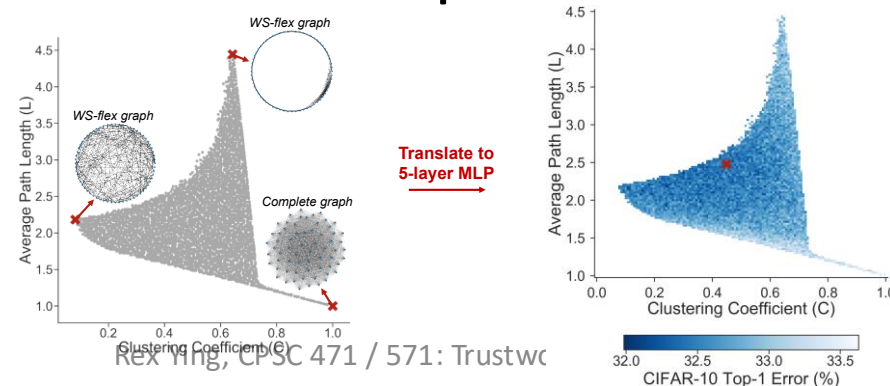
- A novel representation of neural networks: **relational graphs**



- Relational graphs can represent **diverse neural architectures**

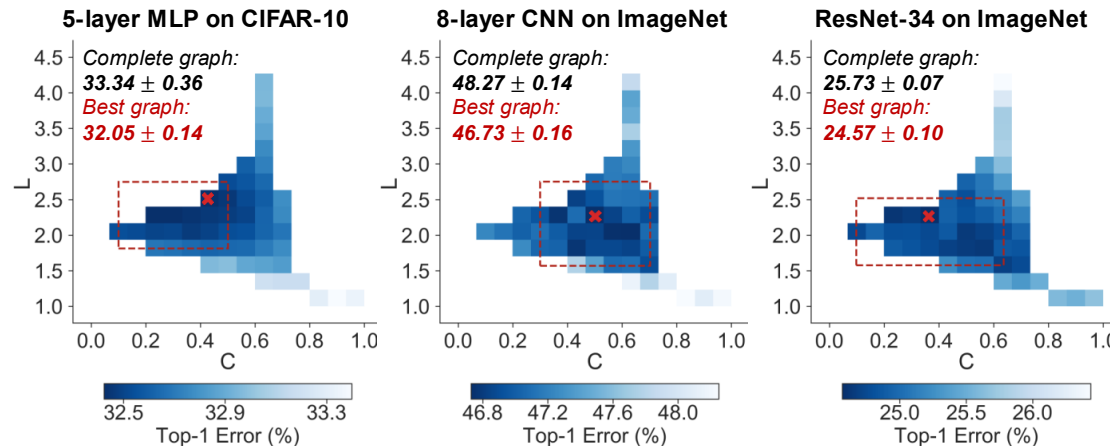


- **Tools from network science** → Graph structure vs NN performance



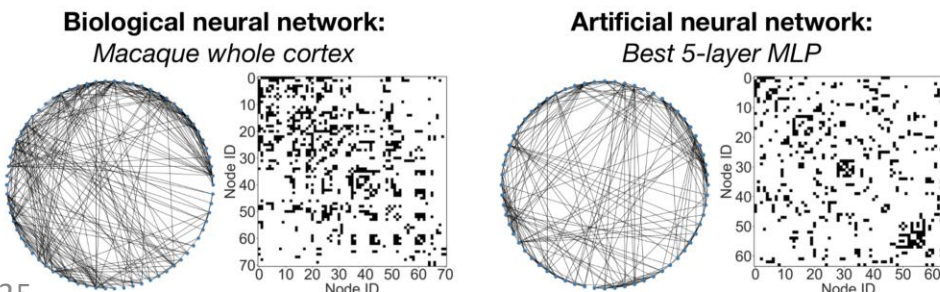
Overview: Key Findings

- **Consistent “Sweet Spot”** for top NNs across architectures



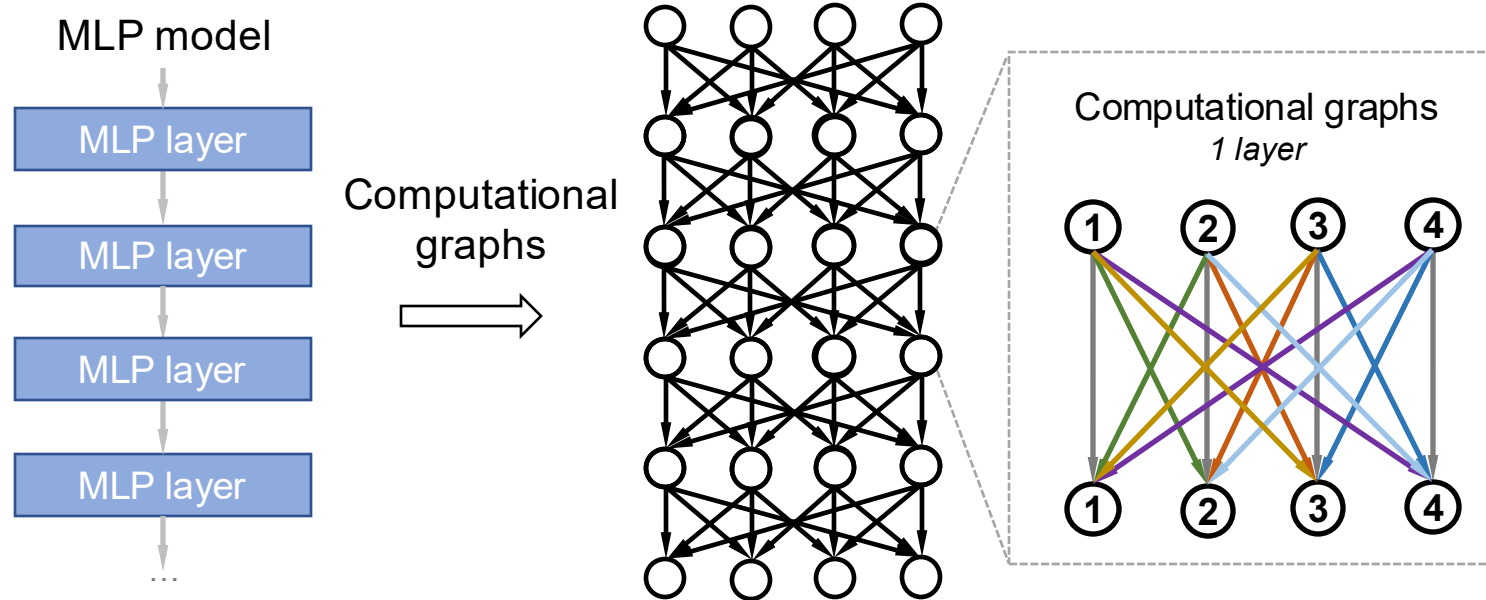
- Graphs with certain structure measures consistently performs well (controlling computational budgets)

- Top artificial NNs are **similar to real biological NNs**

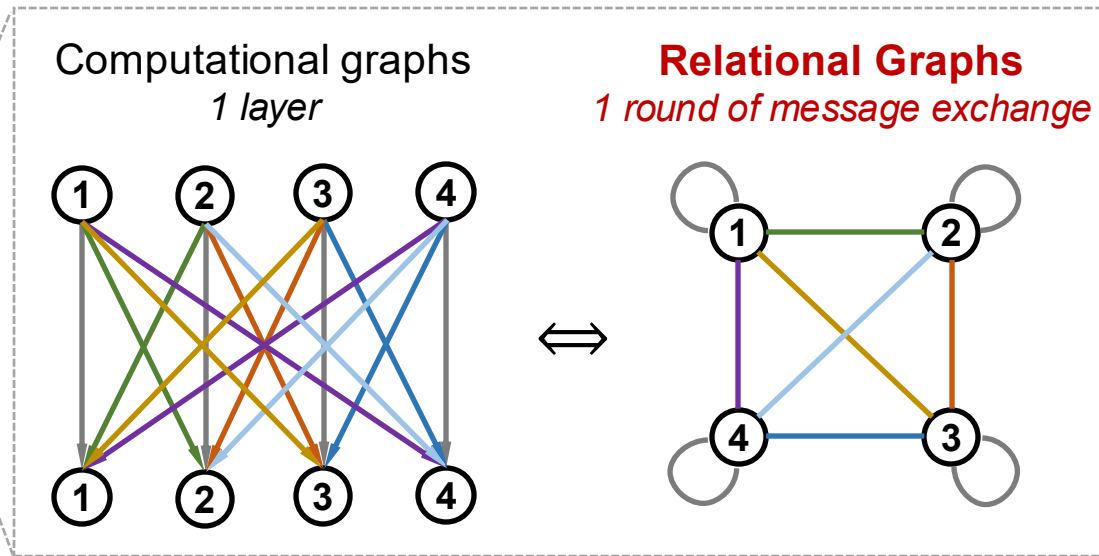
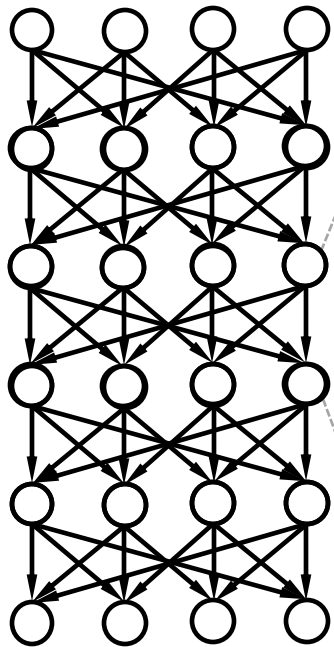


- Graph structure of the best 5-layer MLP we found, is similar to the macaque whole cortex network

NNs as Computational Graphs



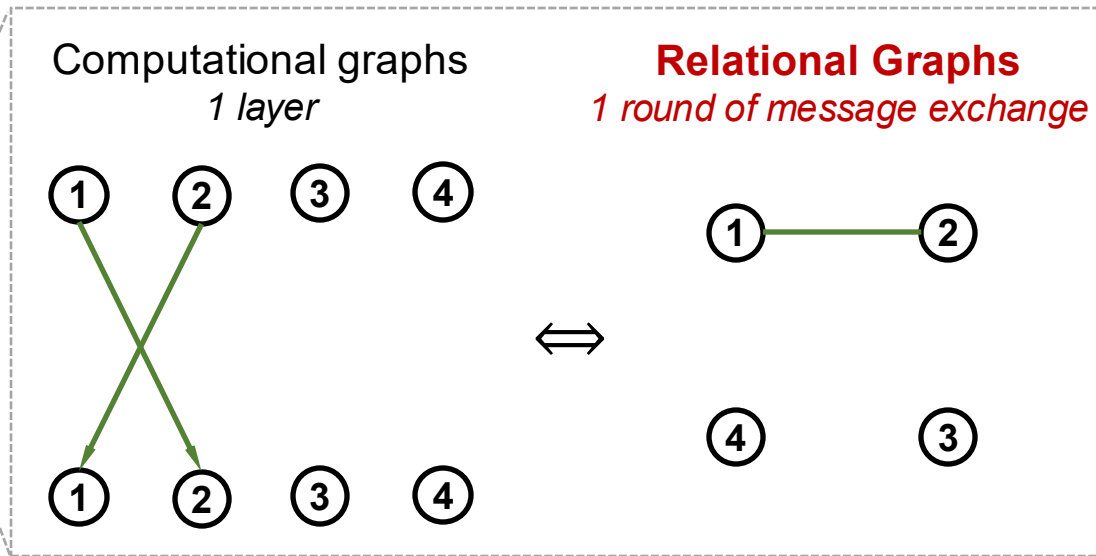
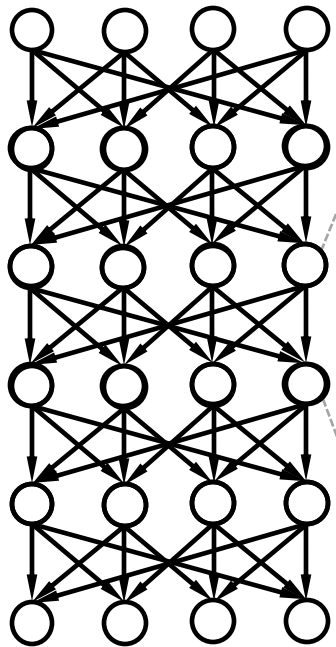
Our approach: Relational Graphs



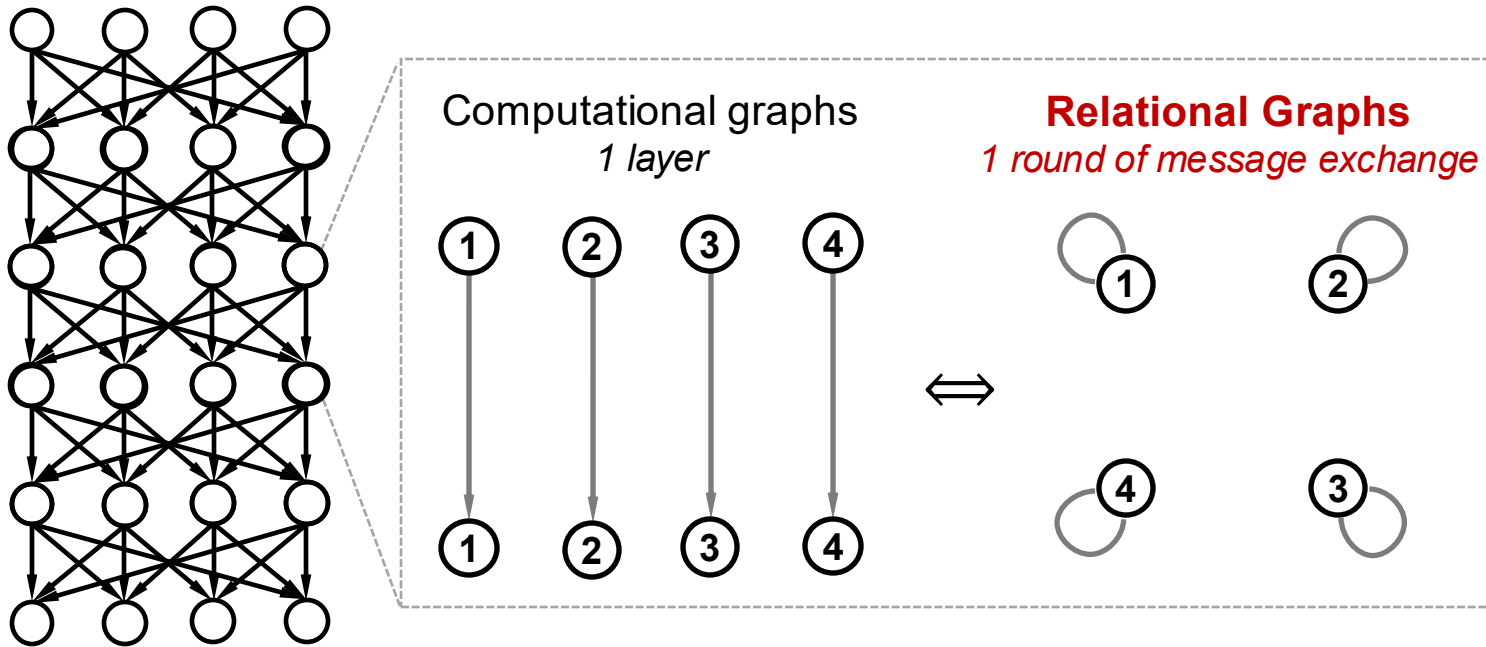
Relational graph definition:

- **Nodes** are neurons
- **Edges** specify (undirected) connectivity between neurons;
- **Computation** is conducted by **message exchange over the graph structure**, where a node exchange messages with its neighbors

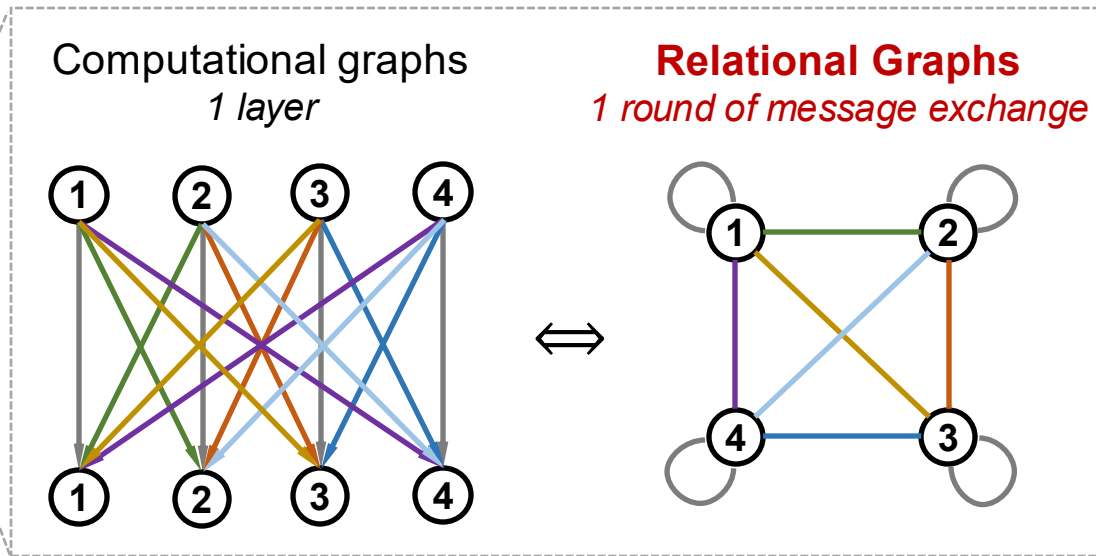
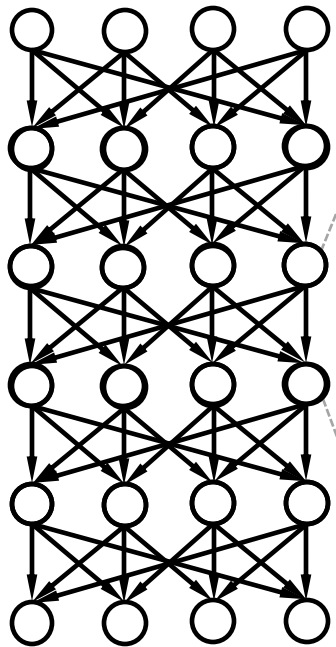
Our approach: Relational Graphs



Our Approach: Relational Graphs



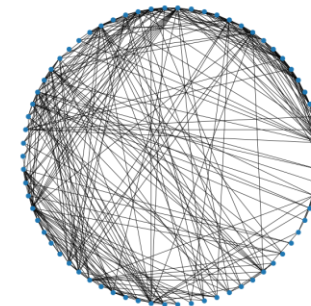
Benefits of Relational Graphs



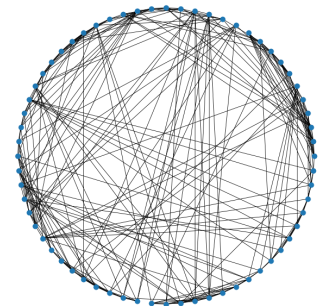
Benefits:

- Flexibility
 - **No restrictions** on graph structure (focus on undirected graph)
- Connections with neuroscience
 - **Bi-directional** information exchange

Biological neural network:
Macaque whole cortex

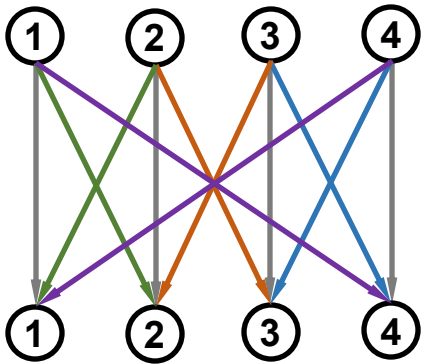


Artificial neural network:
Best 5-layer MLP

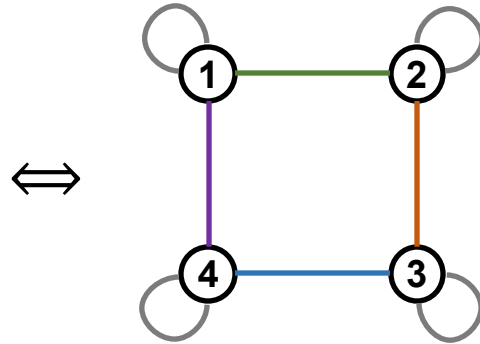


Diverse Relational Graphs

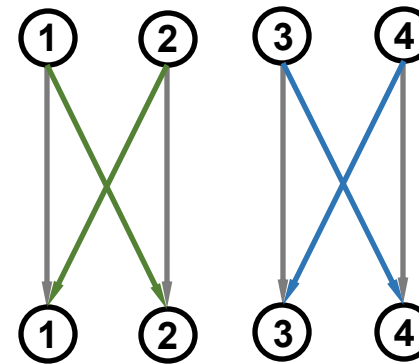
Computational graphs
1 layer



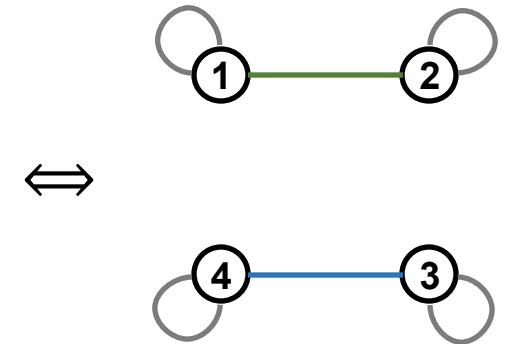
Relational Graphs
1 round of message exchange



Computational graphs
1 layer

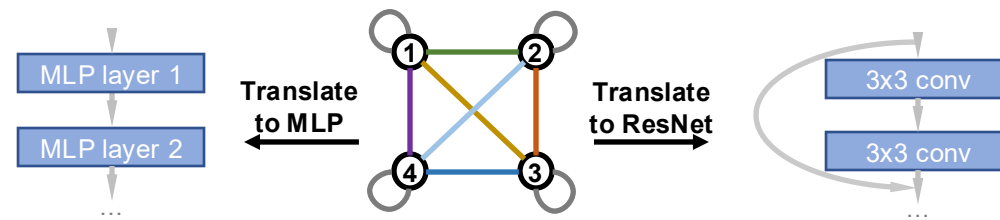


Relational Graphs
1 round of message exchange



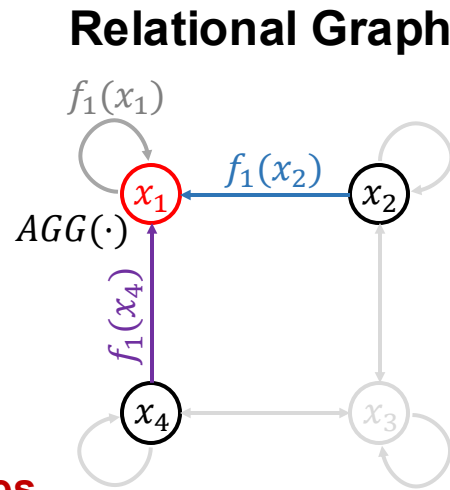
Overview: Methodology

- A novel representation of neural networks: relational graphs
- **Relational graphs can represent diverse neural architectures ←**
 - Can represent **architectures from MLP to ResNet**



- Tools from network science → Graph structure vs NN performance

Relational Graphs → Diverse Architectures

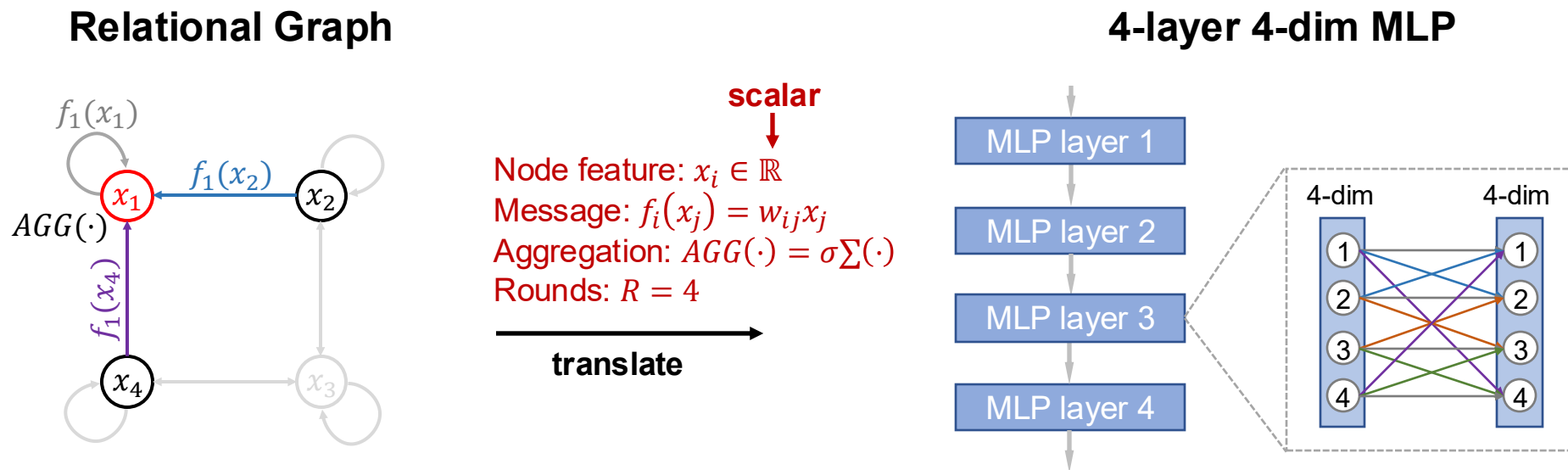


$$\mathbf{x}_v^{(r+1)} = \text{AGG}^{(r)}(\{f_v^{(r)}(\mathbf{x}_u^{(r)}), \forall u \in N(v)\})$$

The same relational graph → diverse architectures

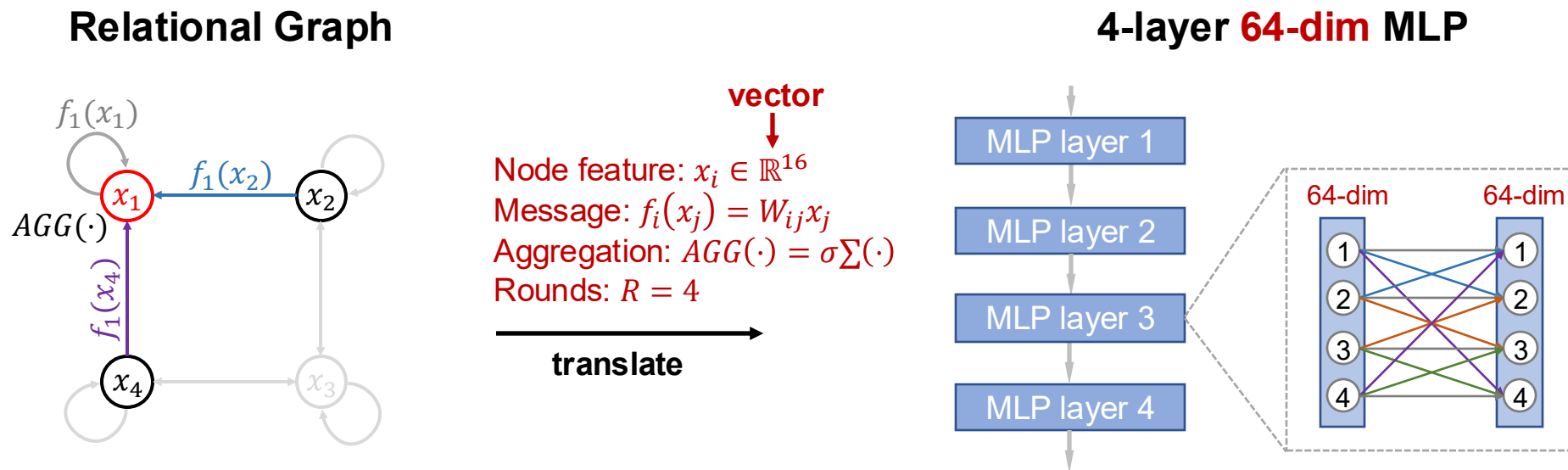
4 Key components	Fixed-width MLP	Variable-width MLP	ResNet-34	ResNet-34-sep	ResNet-50
Node feature $\mathbf{x}_i$	Scalar: 1 dimension of data	Vector: multiple dimensions of data	Tensor: multiple channels of data	Tensor: multiple channels of data	Tensor: multiple channels of data
Message function $f_i(\cdot)$	Scalar multiplication	(Non-square) matrix multiplication	3×3 Conv	3×3 depth-wise and 1×1 Conv	3×3 and 1×1 Conv
Aggregation function $\text{AGG}(\cdot)$	$\sigma(\sum(\cdot))$	$\sigma(\sum(\cdot))$	$\sigma(\sum(\cdot))$	$\sigma(\sum(\cdot))$	$\sigma(\sum(\cdot))$
Number of rounds R	1 round per layer	1 round per layer	34 rounds with residual connections	34 rounds with residual connections	50 rounds with residual connections

Relational Graphs $\rightarrow$ Diverse Architectures



MLP as relational graph

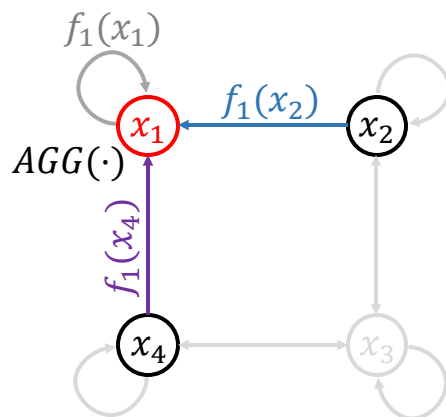
Relational Graphs $\rightarrow$ Diverse Architectures



MLP as relational graph

Relational Graphs $\rightarrow$ Diverse Architectures

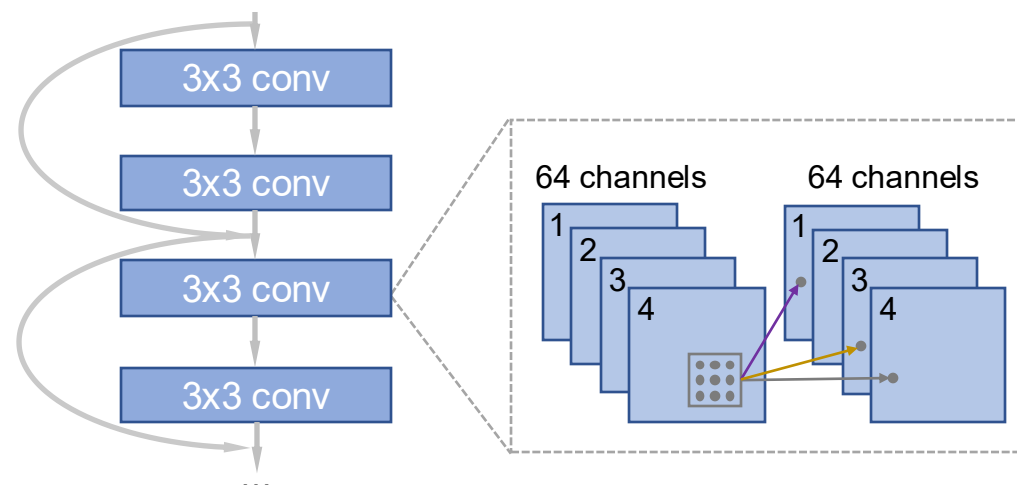
Relational Graph



tensor
Node feature: $x_i \in \mathbb{R}^{16 \times H \times W}$
Message: $f_i(x_j) = W_{ij} * x_j$
 W_{ij} is 3x3 conv kernel
Aggregation: $AGG(\cdot) = \sigma \Sigma(\cdot)$
Rounds: $R = 34$

translate

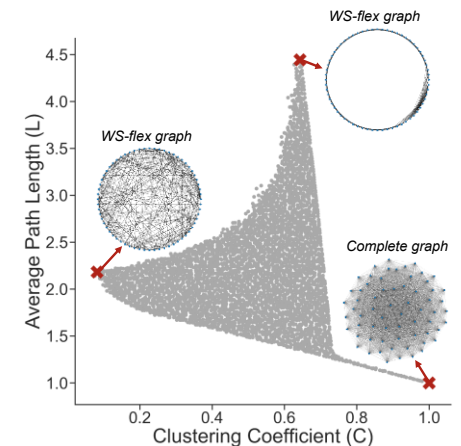
ResNet-34 64-channels



ResNet-34 as relational graph

Overview

- A novel representation of neural networks: relational graphs
- Relational graphs can represent diverse neural architectures
- **Network science → Graph structure vs NN performance ←**
 - **Graph measures** that characterize graph properties
 - **Graph generators** that generate diverse graphs
 - **Control computational budget**



Measuring Graph Structure

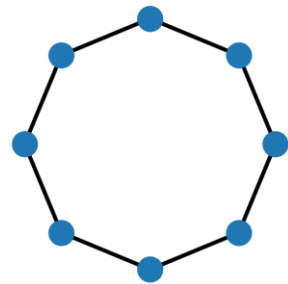
- Graph measures:

- **Global:** average path length (L)

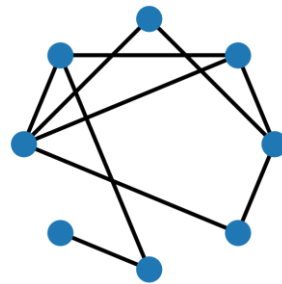
The average shortest path distance between any pair of nodes

- **Local:** clustering coefficient (C)

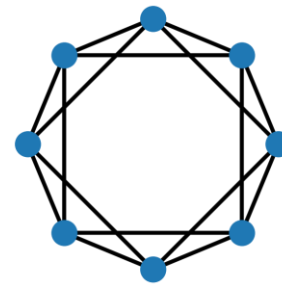
A measure of the degree to which nodes in a graph tend to cluster together



$$L = 2.3$$
$$C = 0$$



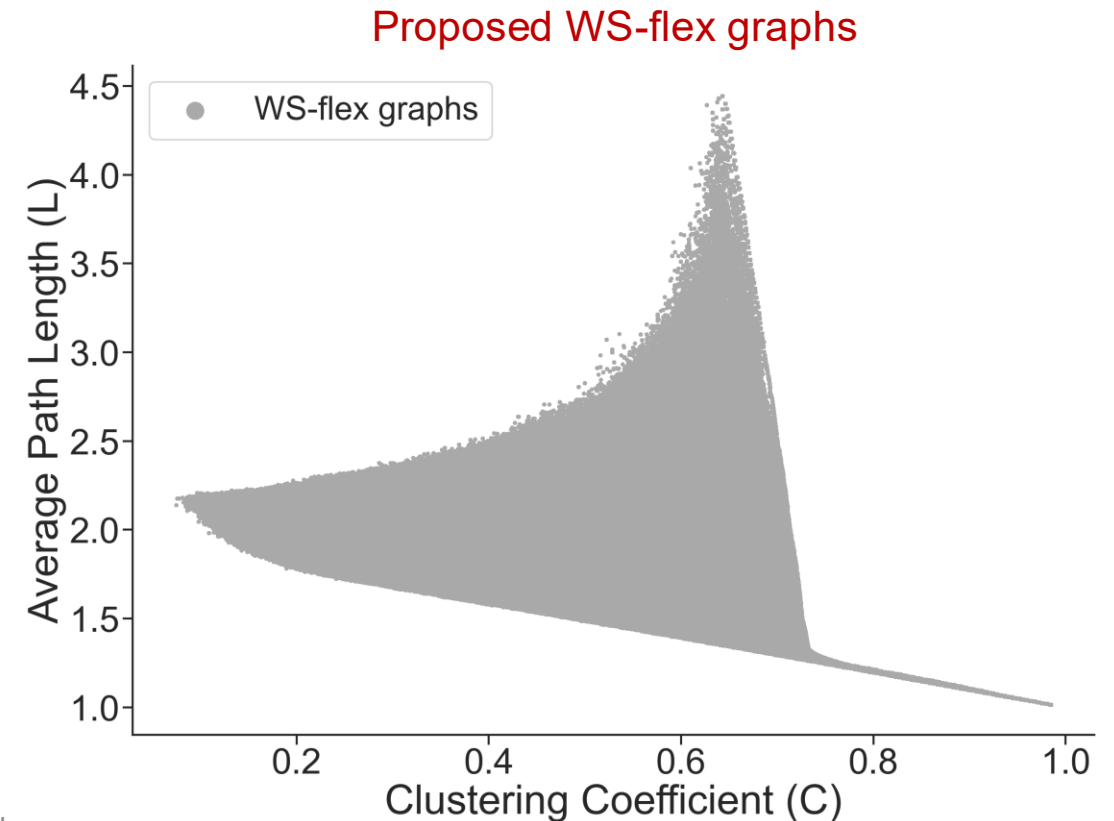
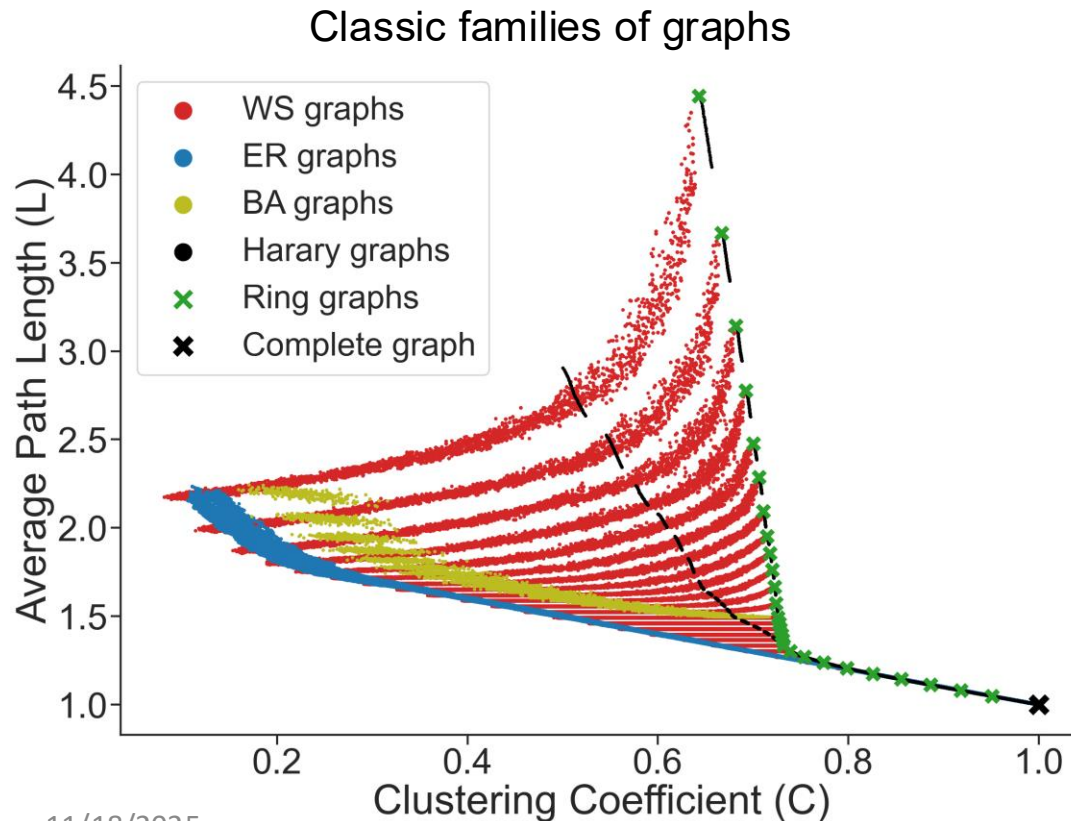
$$L = 2.0$$
$$C = 0.1$$



$$L = 1.4$$
$$C = 0.5$$

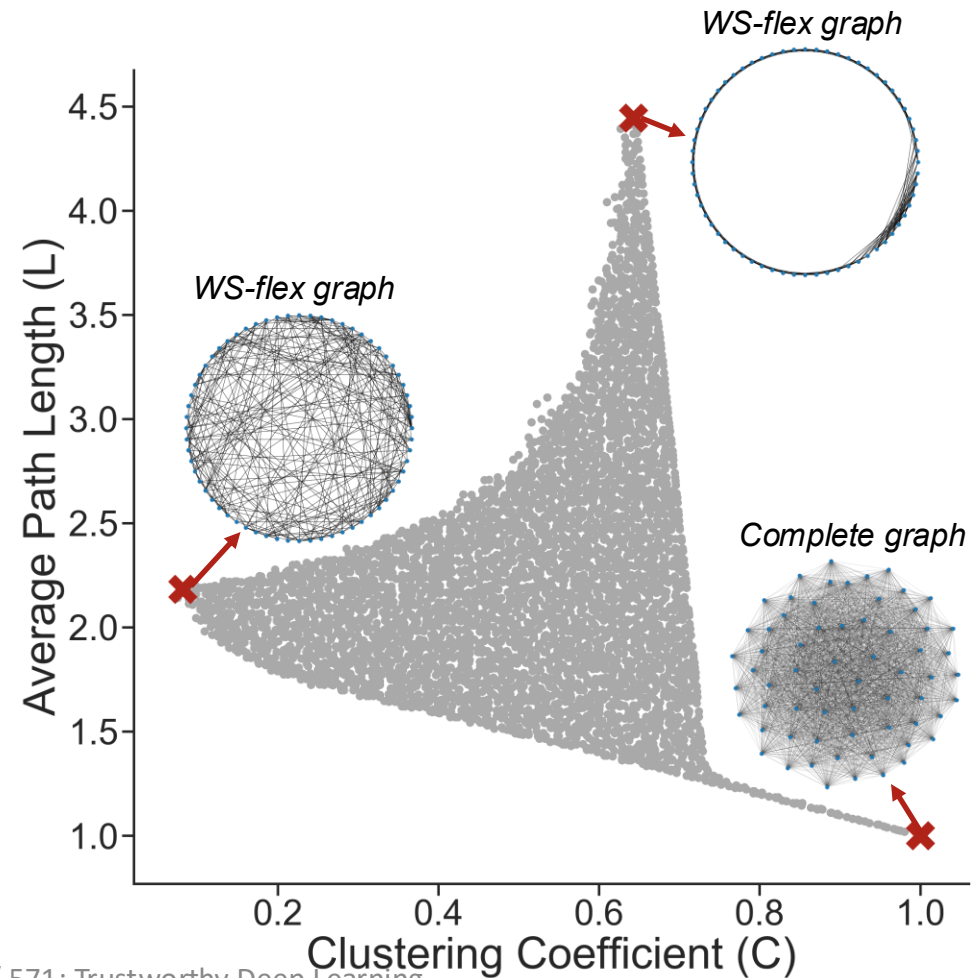
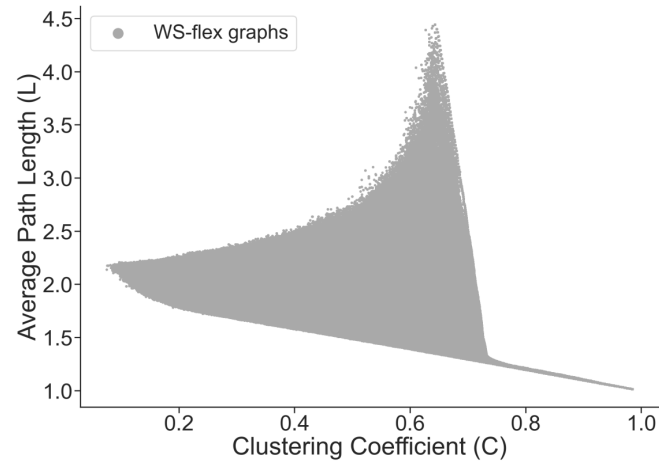
Generating Diverse Graphs

- WS-flex graphs have a much better coverage



Generating Diverse Graphs

- Visualization of WS-flex graphs



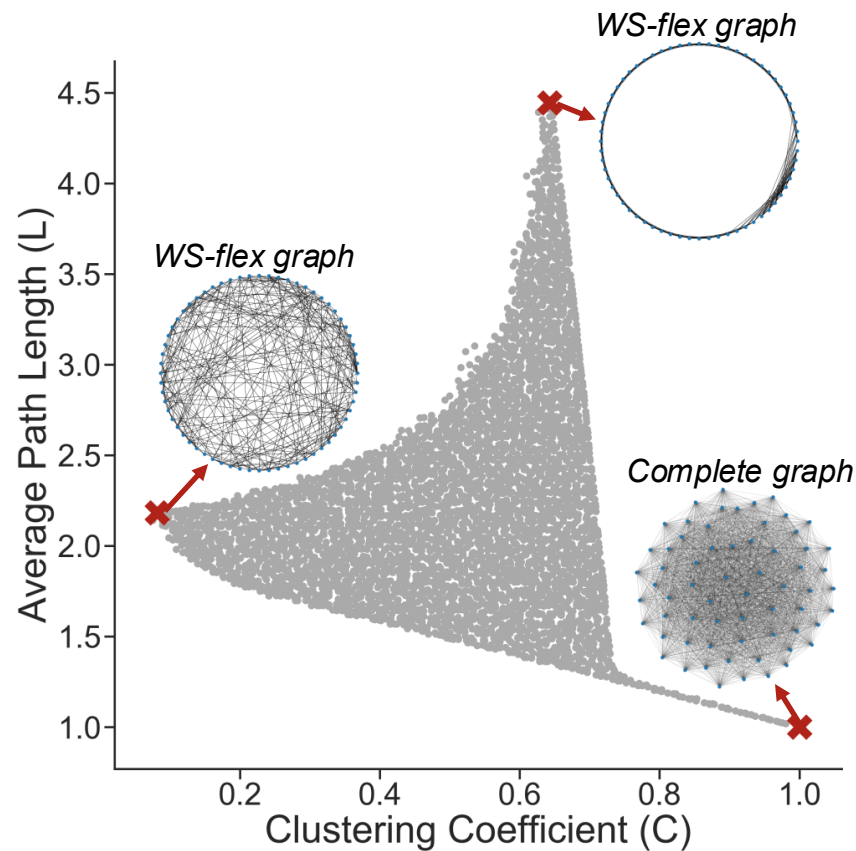
Experimental Setup

- 5-layer 512-dim **MLPs** on **CIFAR-10**
 - 3942 graphs, results averaged over 5 seeds
- **CNNs & ResNet families & EfficientNet-B0** on **ImageNet**
 - 52 graphs per experiment, results averaged over 3 seeds
- Computational budgets in all experiments are controlled

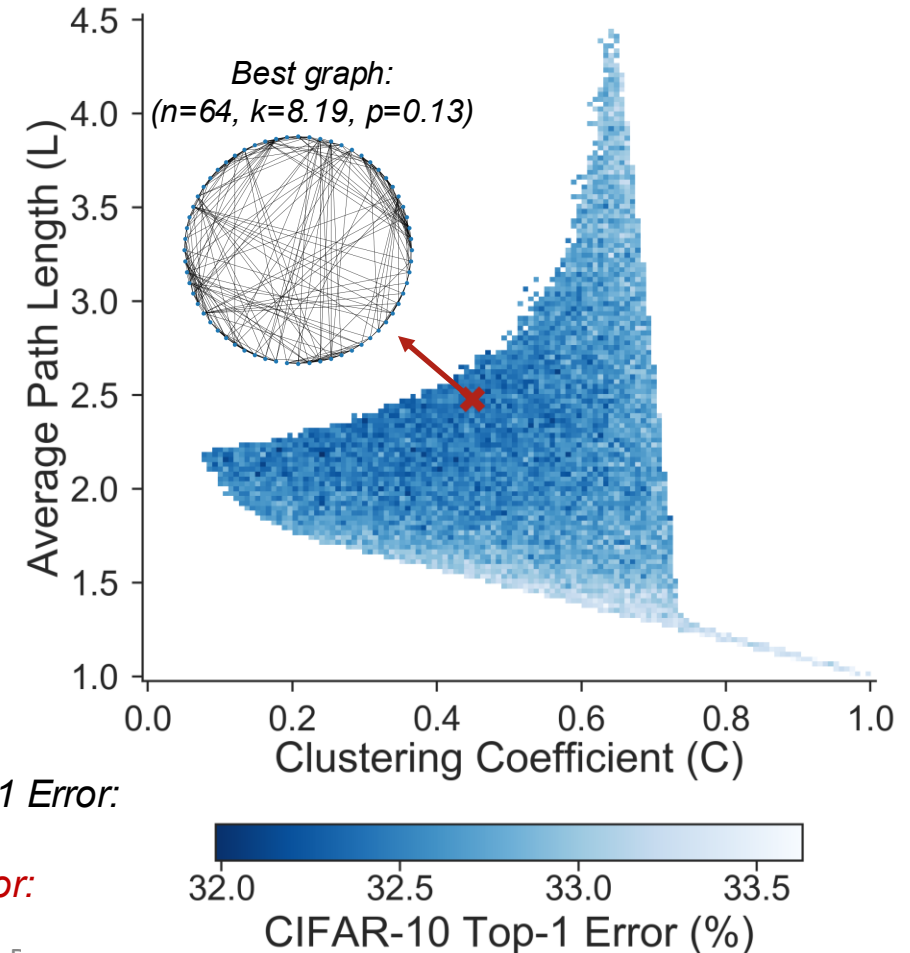
Overview: Findings

- **Finding 1: Consistent Sweet Spot** for top NNs across architectures
- **Finding 2:** NN Performance as a **smooth function** over graph measures
- **Finding 3:** Sweet spot can be **quickly identified**
- **Finding 4:** Top artificial NNs are **similar to real NNs**

Finding 1: Sweet Spot for Top NNs



Translate to
5-layer MLP



Complete graph Top-1 Error:

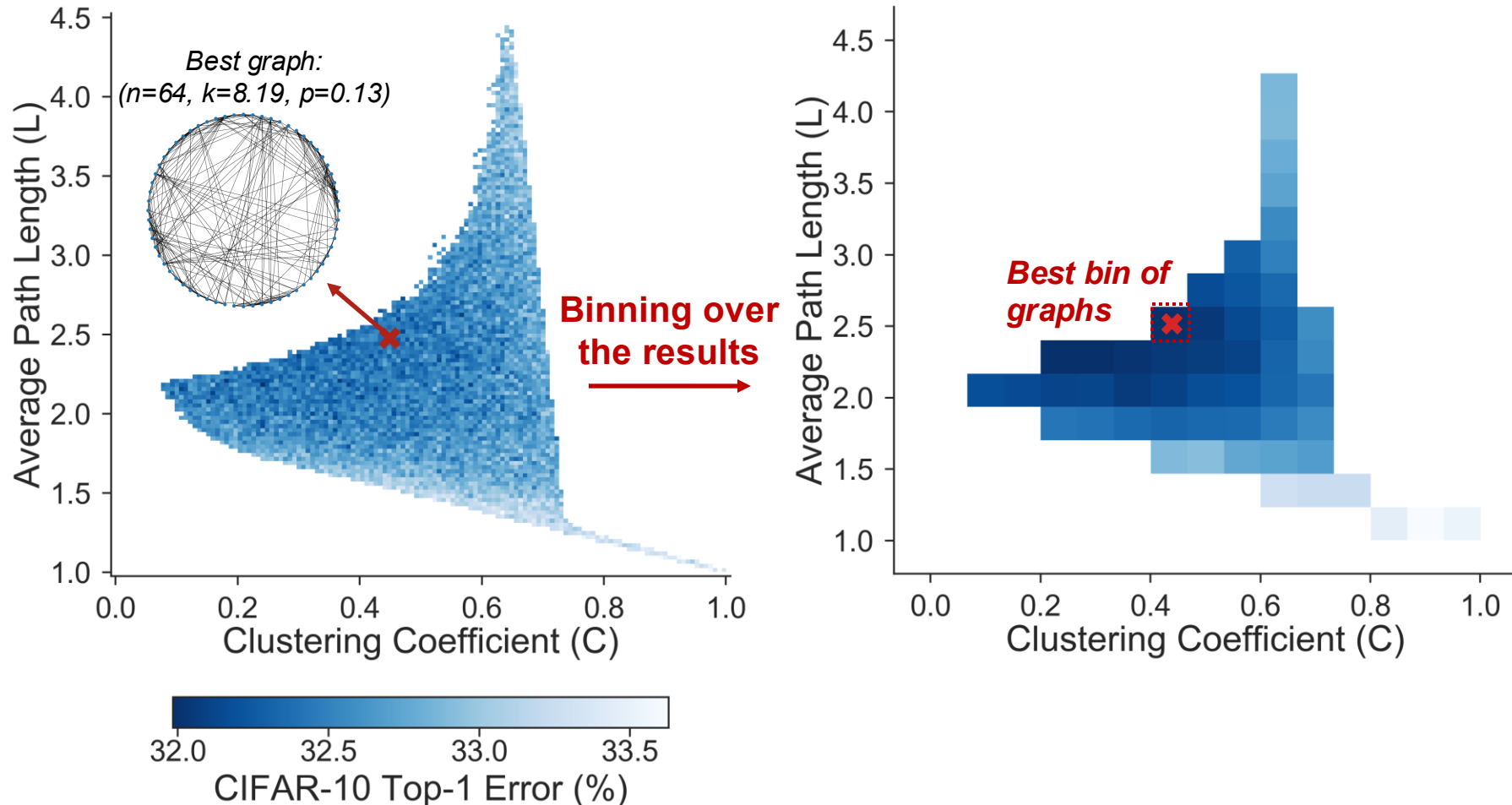
33.34 ± 0.36

Best graph Top-1 Error:

32.05 ± 0.14

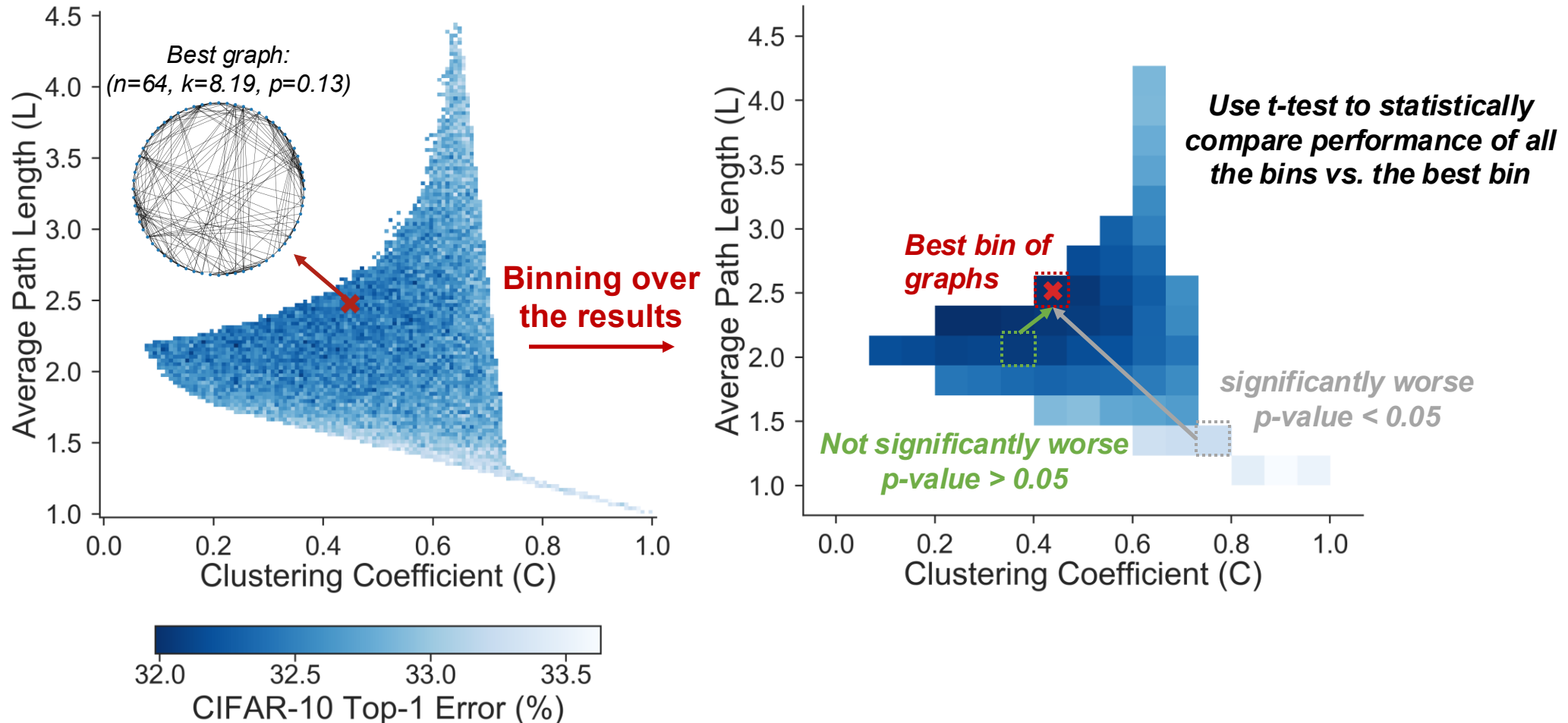
Finding 1: Sweet Spot for Top NNs

5-layer MLP on CIFAR-10



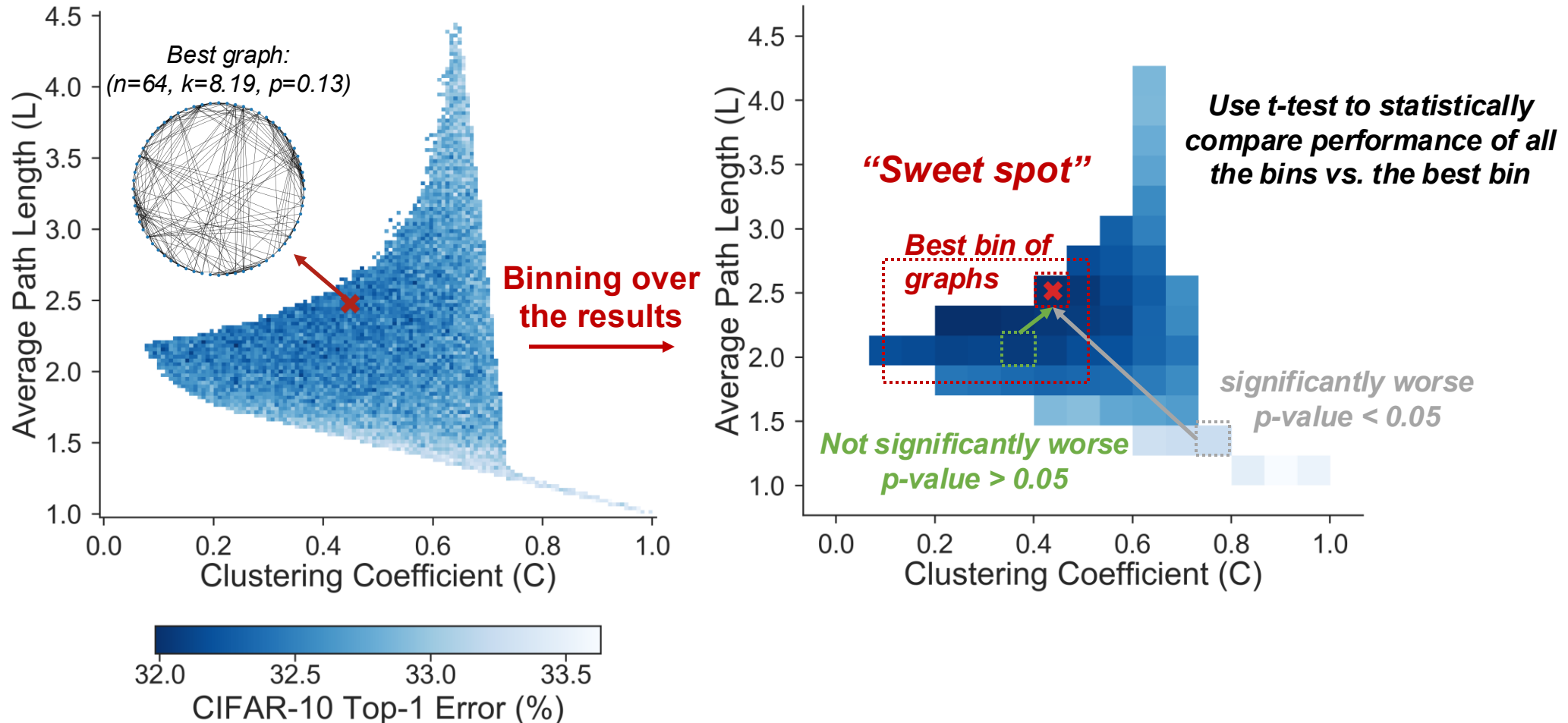
Finding 1: Sweet Spot for Top NNs

5-layer MLP on CIFAR-10

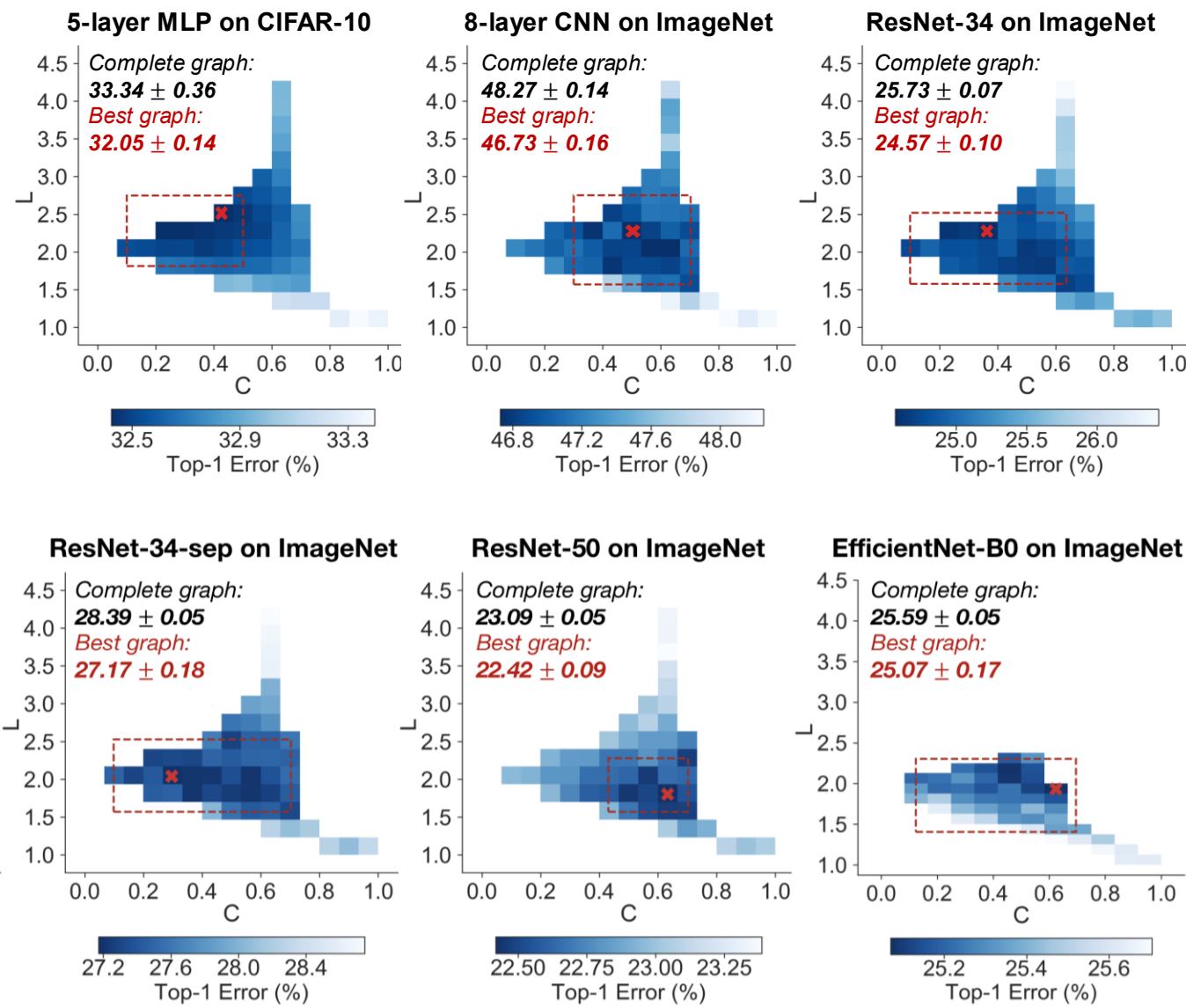


Finding 1: Sweet Spot for Top NNs

5-layer MLP on CIFAR-10



Finding 1: Consistent Sweet Spot for Top NNs



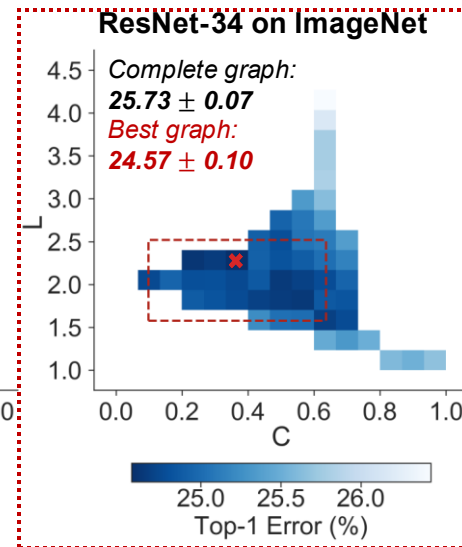
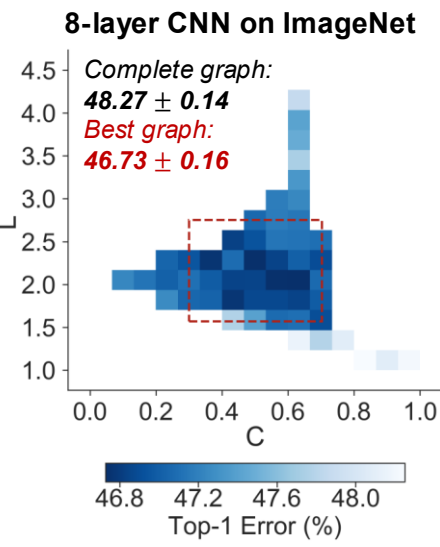
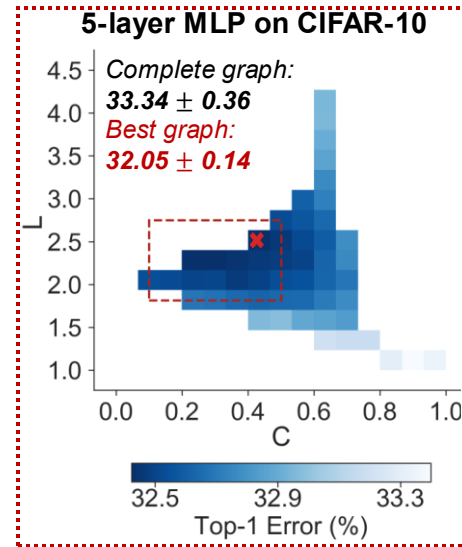
A consistent sweet spot

$$C \in [0.43, 0.50]$$

$$L \in [1.82, 2.28]$$

Finding 1: Consistent Sweet Spot for Top NNs

2.89e6 FLOPS



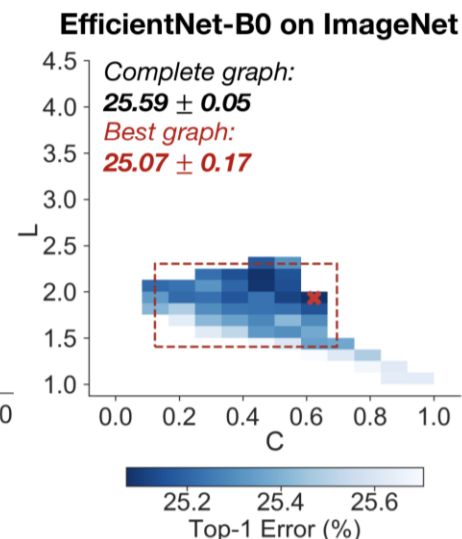
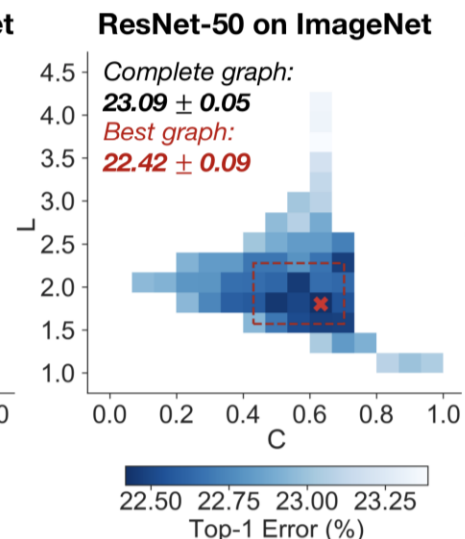
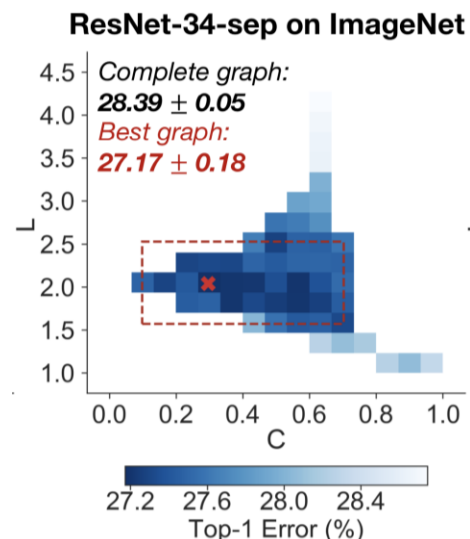
3.66e9 FLOPS

~1000x FLOPS difference
But similar sweet spot!

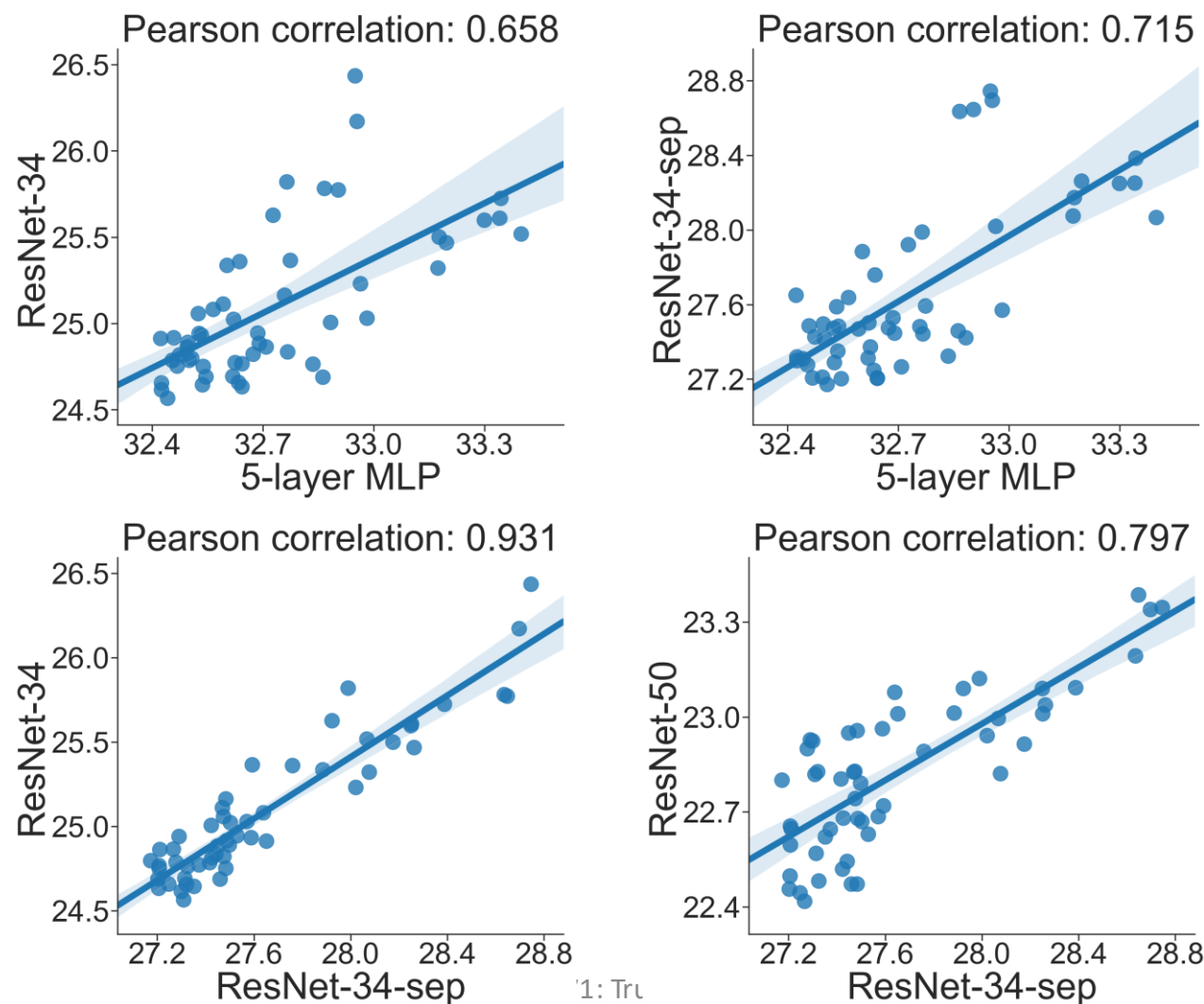
A consistent sweet spot

$C \in [0.43, 0.50]$

$L \in [1.82, 2.28]$



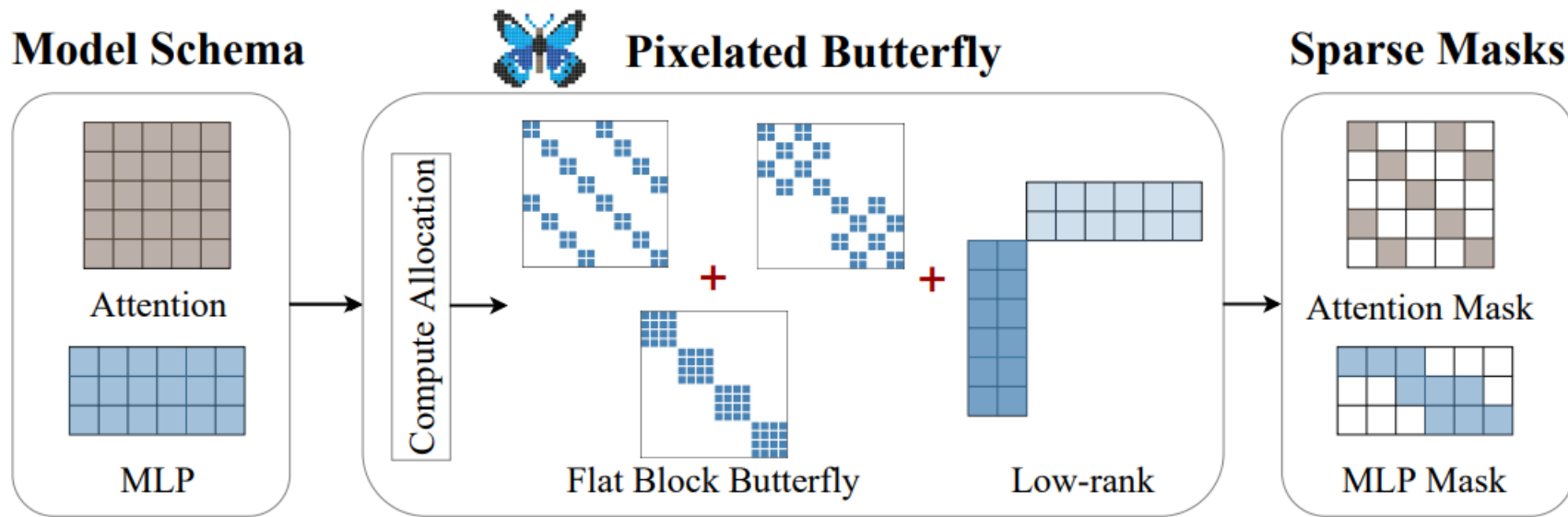
Finding 1: Consistent Sweet Spot for Top NNs



Quantitative
consistency across
architectures

Alternative Sparse Patterns

- Butterfly networks
- **Caveat:** Even with a very specialized CUDA kernel, butterfly matrix multiply ($O(n \log n)$ complexity) is only faster than dense matrix for large values of n

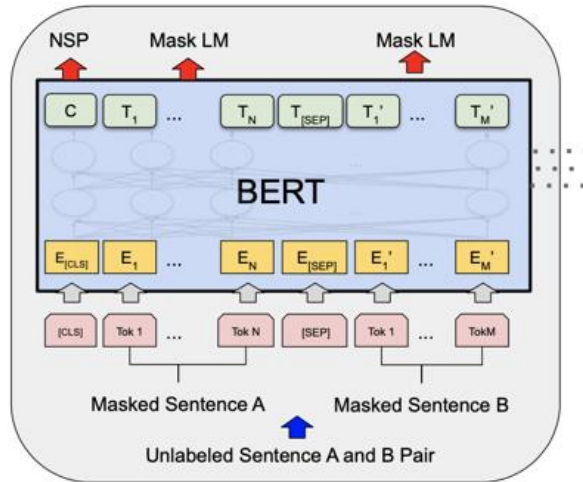


Content

- Distillation
 - Model distillation
 - Data distillation
- Quantization
- Sparse Neural Networks
- Sparse Attention

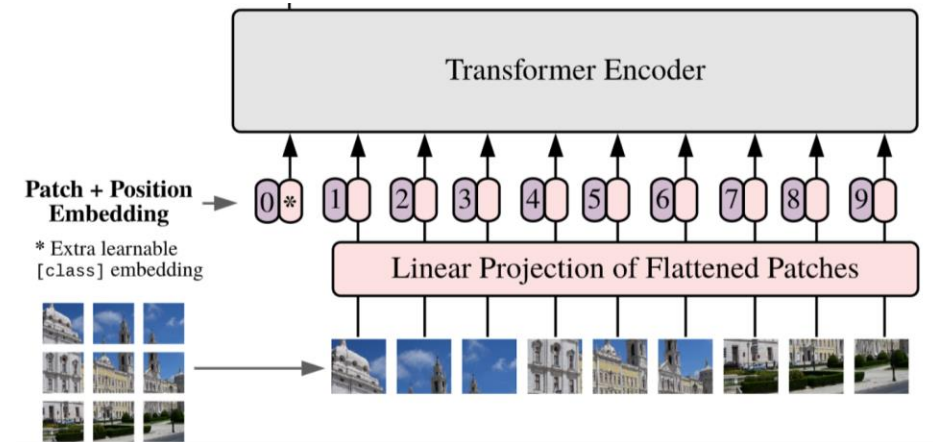
Compression

Transformers Changed Deep Learning



- NLP Transformers

- Most downloaded pretrained models in *Huggingface* are BERT (18.9 M) and GPT2 (11.1 M).

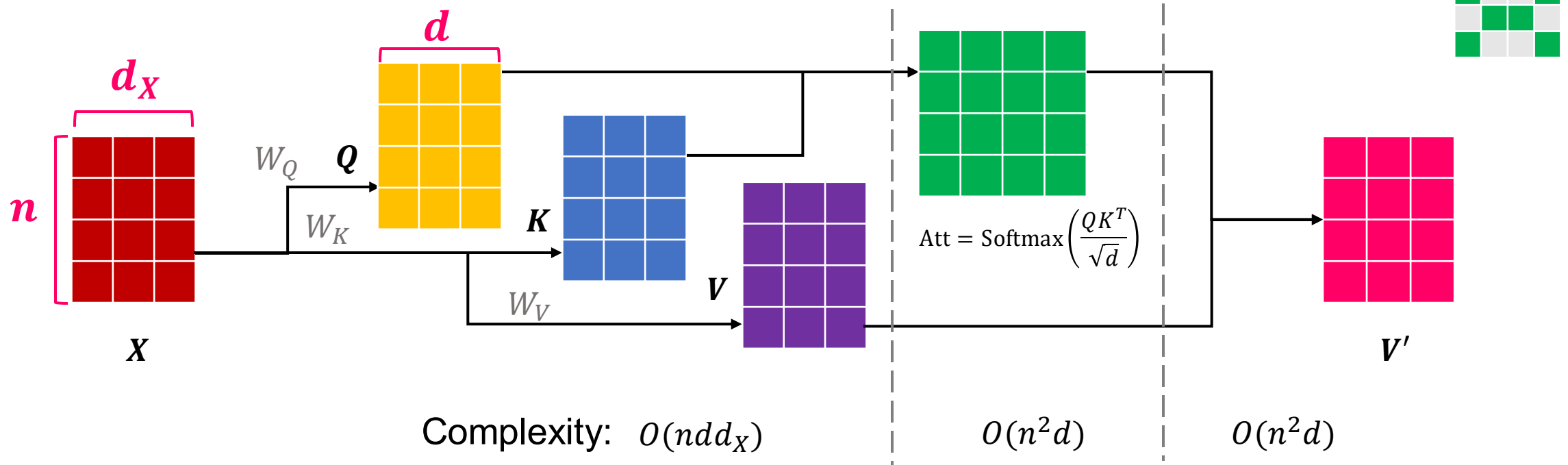


- CV Transformers

- On ImagerNet, ViT achieves 90.4% accuracy, beating CNN-based model (Inception 78.3%, ResNet 80.9%...)

Attention Mechanism — a closer look

- For an input sequence with n tokens and dimension d_x



The computation complexity is quadratic to number of tokens n

Scaling to long sequences

- Popular transformers are designed for relatively **short sequence**:
 - BERT/GPT take **512** tokens as the input sequence.
 - ViT take **197** tokens in the input sequence (with 16×16 pixels as a patch)
- Long sequence scenario:
 - The median length of a research paper is **4,133** words (data from PubMed)
 - Byte-level NLP task
 - Finer-grained image patch: 8×8 patch->**785** tokens; 4×4 patch-> **3137** tokens
- Trouble from **quadratic complexity**

BERT input length	512	1024	2048	3072	4096
GPU memory (GB)	1.9	4.5	12.5	23.6	44.2
Per-iteration time (s)	0.12	0.20	0.24	0.79	1.98



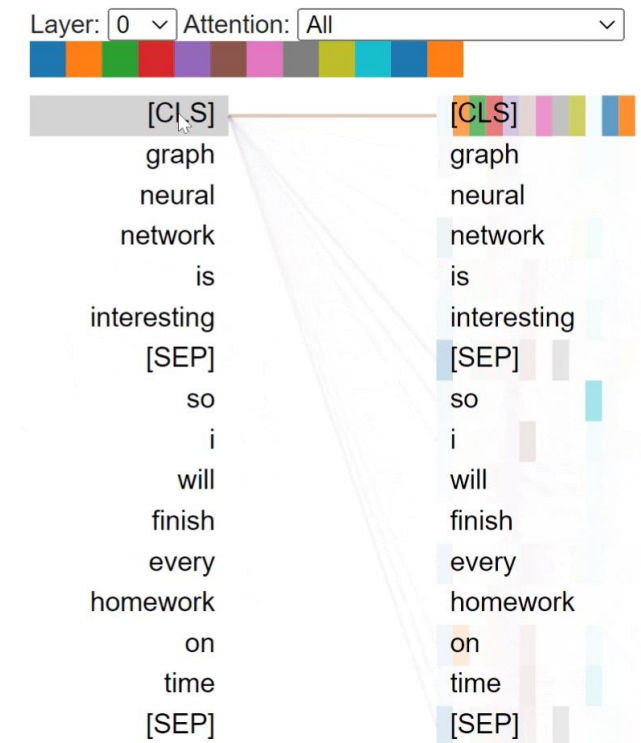
~ 20×

Full attention is not necessary

- **Observation 1:** Although every attention is calculated, most of them are close to 0, the resulting attention maps are usually **sparse**.
- **Observation 2:** non-zero attention mostly appear between the node and its **local neighbors**. (**local attention**).
- **Observation 3:** some **key words** like “so” almost attend to every token in the sentence. (**global attention**)

We can simplify the self-attention (full-attention) with appropriate sparse pattern

A sentence encoded by pretrained BERT

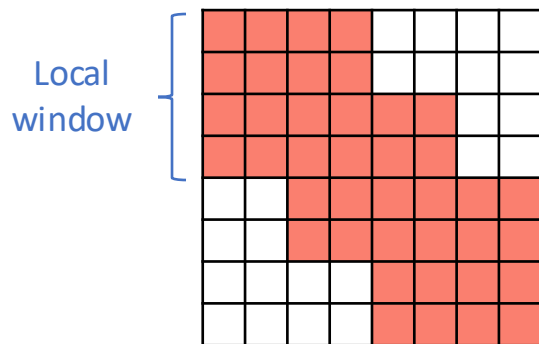


Attention between pairwise tokens

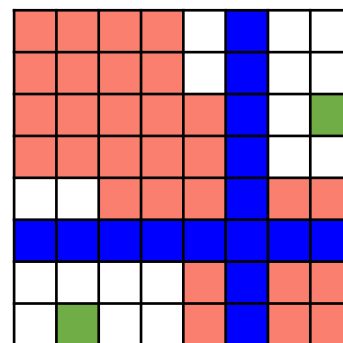
Existing Sparse Transformers

- Masking attention (with sparse pattern) can reduce complexity from $\mathcal{O}(n^2)$ to $\mathcal{O}(n)$.
- Previous sparse transformers design **attention patterns** to approximate the full attention.

Attention Patterns (Masks)



Longformer [Beltagy et al., 2020]



BigBird [Zaheer et al., 2020]

- No attention
- **Local attention**: tokens attend within a local window (size = 4 in the figure)
- **Global attention**: one global token attend to all tokens
- **Random attention**: randomly select attentions

Limitations of Sparse Transformers

- **Inaccurate approximation of full-attention**

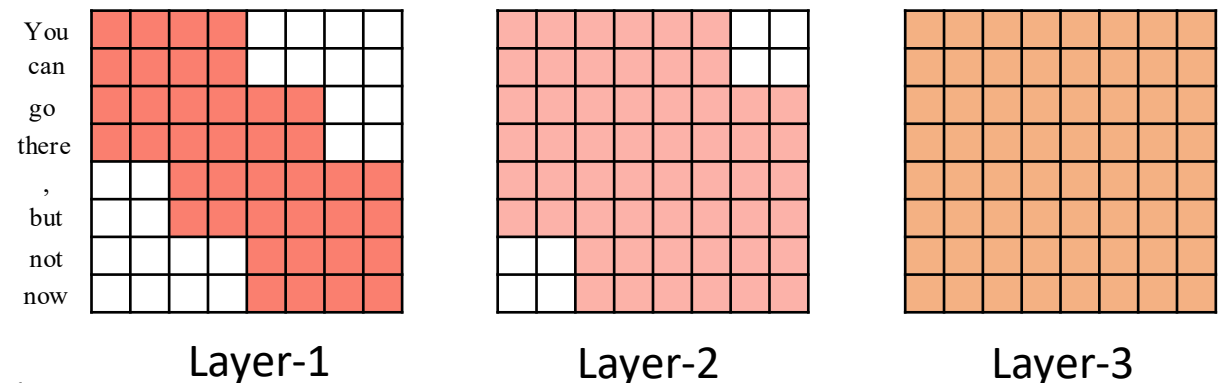
- Based on assumption about attention structure (low-rank, proximity, ...)

- **Slow information propagation**

- To obtain attention between disconnected pair of tokens, have to **stack multiple layers** to extend the attention receptive field
- Examples: attention between “You” and “not” needs **3** transformer layers

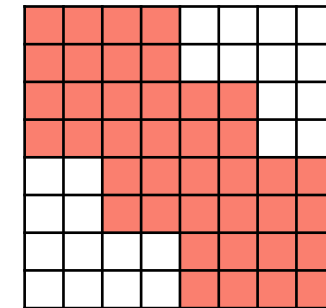
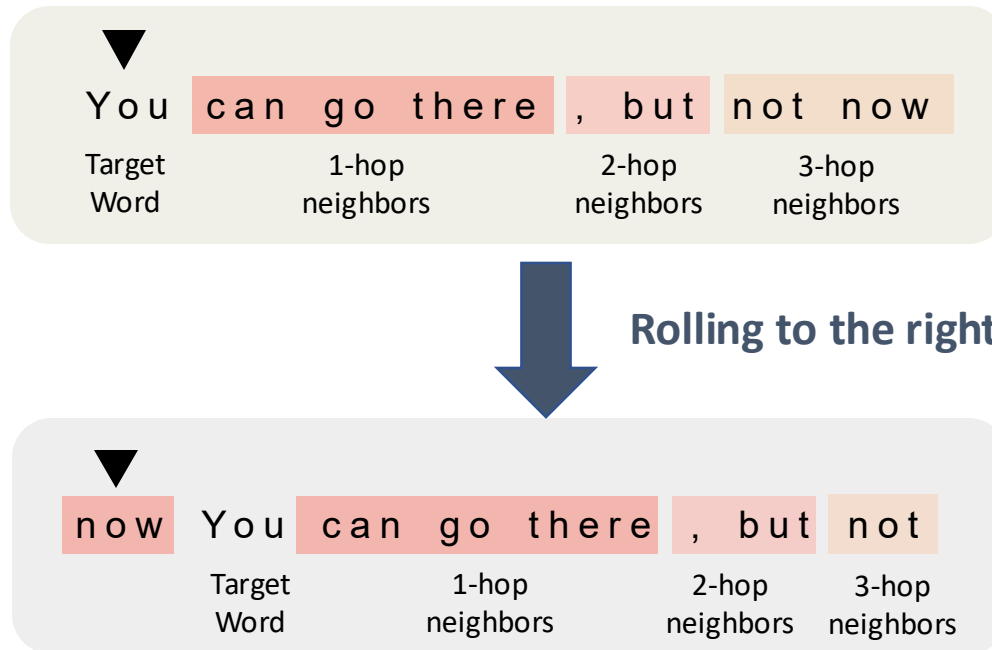


Equivalent attention receptive field



Limitations of Sparse Transformers

- Not robust to input perturbations



Fixed attention pattern

Attention structure changes: "now" changes from 3-hop to 1-hop neighbor
(Full-attention does not have such problem)

Sparse Transformers — Reformer [Kitaev et al., 2020]

- Reformer is based on **locality sensitive hashing (LSH)** – a nearest-neighbor algorithm
 - **LSH** Definition: A hashing is called **locality-sensitive** if nearby vectors get the same hash with high probability and distant ones do no.
 - Main idea: First **grouping** tokens into different buckets (using LSH), then calculating **attentions within the same bucket**
 - The complexity is reduced to $O(b)$, b is the bucket size.

Computing QK^T is expensive; but what we really need is $\text{Softmax}(QK^T)$!

Sparse Transformers — Reformer [Kitaev et al., 2020]

- Details of **locality sensitive hashing (LSH)**

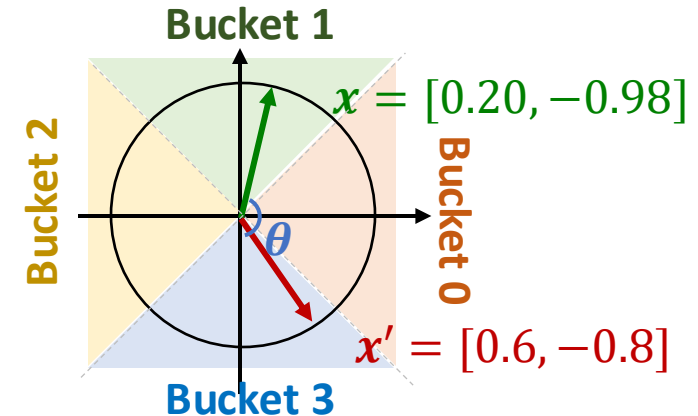
- To get b hashes (buckets), introducing a fixed random matrix $R \in \mathbb{R}^{d \times \frac{b}{2}}$, the hash of input token $\mathbf{x} \in \mathbb{R}^d$ is computed as

$$h(\mathbf{x}) = \operatorname{argmax}([\mathbf{x}R; -\mathbf{x}R])$$



Element-wise argmax

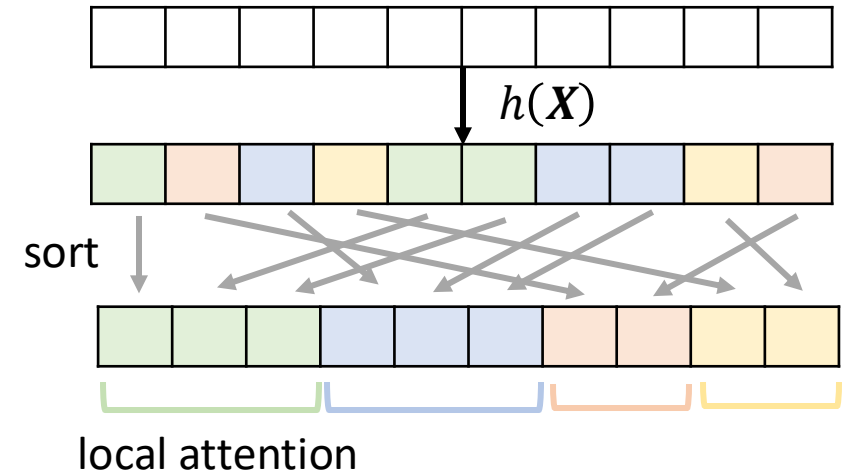
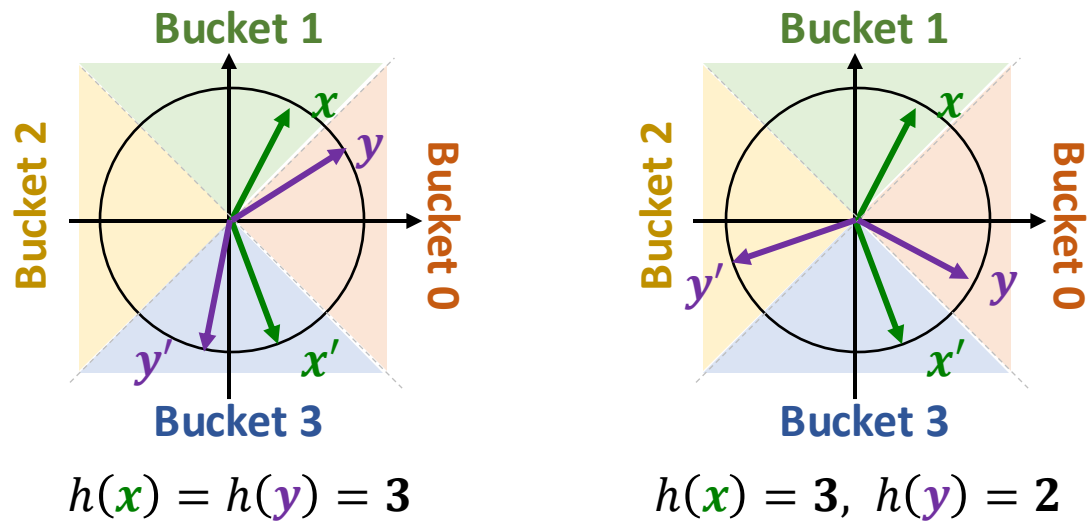
- Setting $d = 2$, $\mathbf{x}$ becomes a 2-d vector
- Setting $b = 4$, we get 4 buckets (0, 1, 2, 3)
- For $\mathbf{x}' = R\mathbf{x}$, if we project $\mathbf{x}$ and $\mathbf{x}'$ to the unit circle, the function of $R \in \mathbb{R}^{2 \times 2}$ is rotation, i.e., $\mathbf{x} \xrightarrow{\text{rotated by } \theta} \mathbf{x}'$



$$h(\mathbf{x}) = \operatorname{argmax}([0.6, -0.8, -0.6, \mathbf{0.8}]) = \mathbf{3}$$

Sparse Transformers — Reformer [Kitaev et al., 2020]

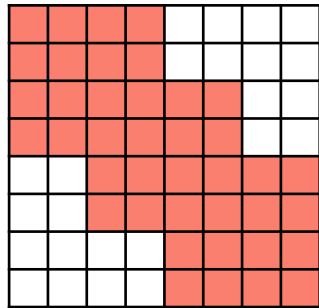
- If two tokens $\mathbf{x}$, $\mathbf{y}$ are close to each other (regarding e.g., angular distance), they have larger probabilities to share the same bucket.



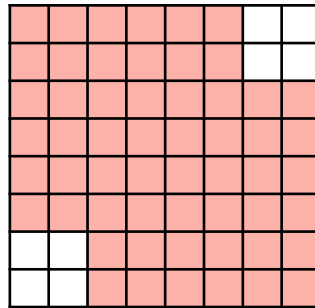
- Attention in Reformer: only calculating attention within the same buckets

Attention Diffusion

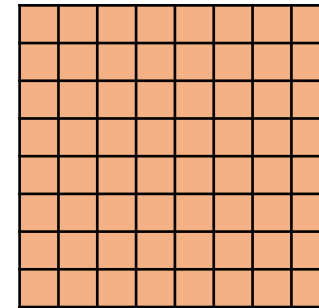
- **Diffuser** propose to **augment sparse transformers** with **attention diffusion** targeting the limitations.
- Calculated attention scores between **directly connected** pairs diffuse to **indirectly connected** pairs, through multiple diffusion steps, **within one layer**.



1-step
diffusion



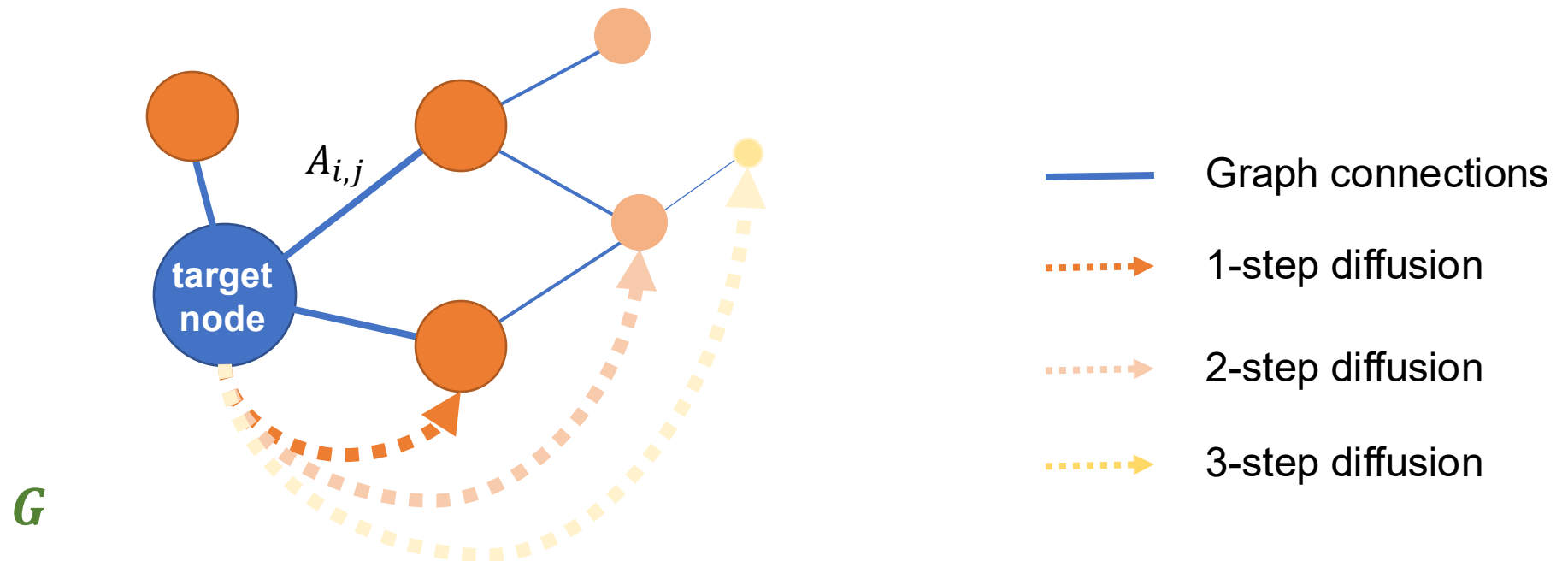
2-step
diffusion



3-step
diffusion

Attention Diffusion — a graph view

- Relationship to **graph diffusion**: By treating the attention matrix A ($A_{i,j}$ is the attention between token i and j) as the adjacency matrix of a graph G , attention diffusion is equivalent to diffusion on G



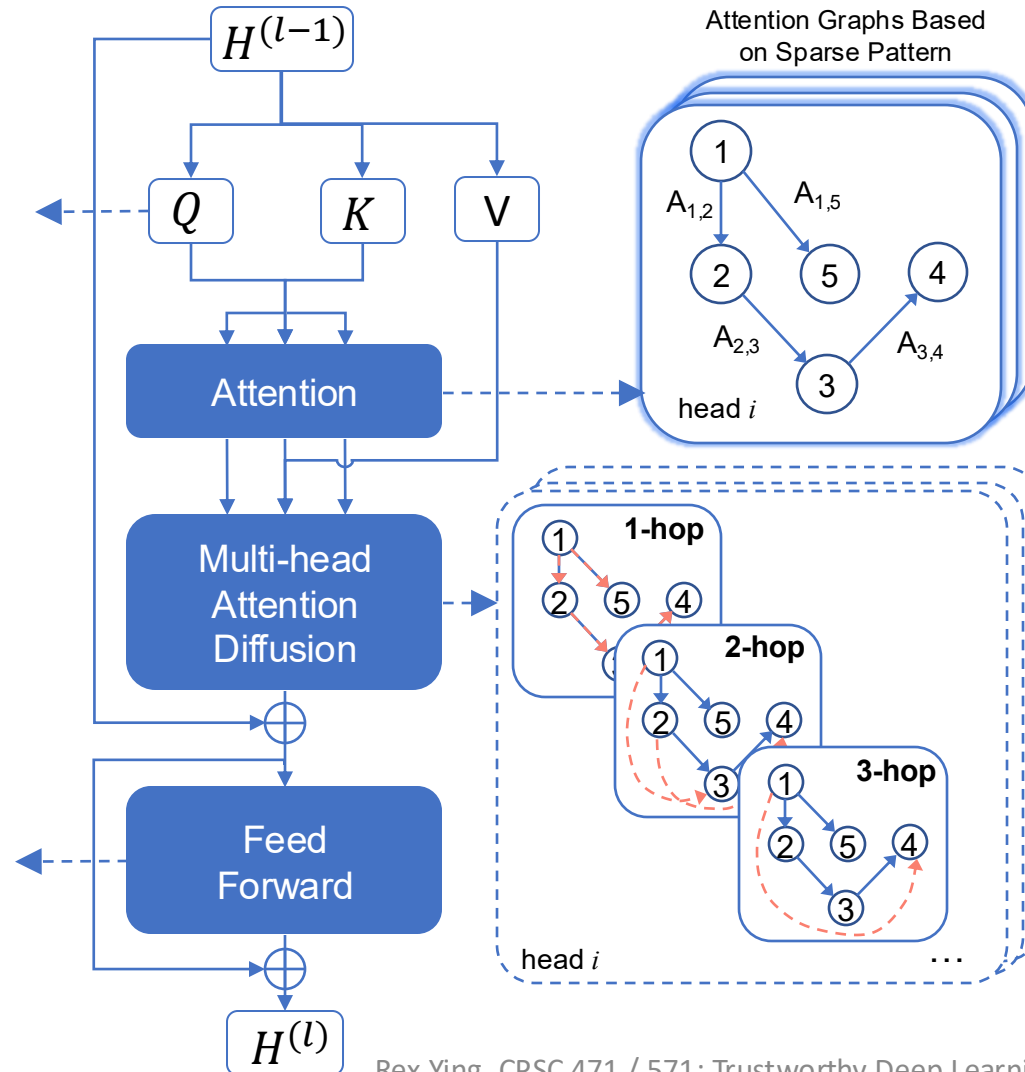
Attention Diffusion — calculations

- Standard attention
 - **Output of attention:** $\mathbf{Z} = \mathbf{A}V$
 - Attention scores: $\mathbf{A}_{i,j} = \frac{\exp(Q_i K_j / \sqrt{d})}{\sum_{j \in \text{Ne}(i)} \exp(Q_i K_j / \sqrt{d})}$
 - V, Q, K are linearly projected from input X
- Attention diffusion:
 - **Output of attention:** $\mathbf{Z} = \mathcal{A}V$
 - Diffusion matrix $\mathcal{A} = \sum_{i=0}^{\infty} \alpha(1 - \alpha)^i \mathbf{A}^i$ (α : teleport probability)
 - Calculate with power iteration:
 - $Z_{(0)} = V$
 - $Z_{(k+1)} = (1 - \alpha)\mathbf{A}Z_{(k)} + \alpha V$
 - Final output: $\mathbf{Z} = Z_{(K)}$, with K diffusion steps

$$K \rightarrow \infty \Rightarrow Z_{(K)} \rightarrow \mathcal{A}V$$

Diffuser — model architecture

- Linear projections to Q, K, V



- Calculate **attention scores** between connected pairs, according to **the sparse pattern**
- Perform **attention diffusion** with power iteration to update V

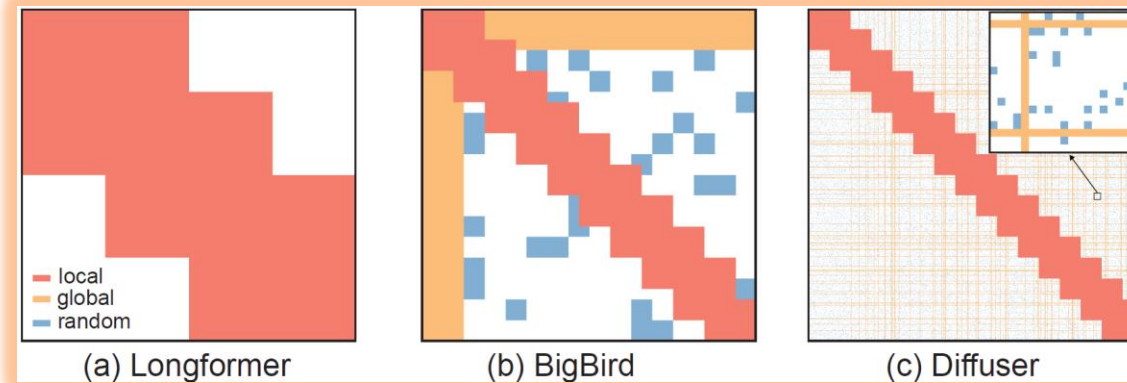
—→ sparse connections
- - -→ information flow

- Postprocessing

Diffuser — sparse pattern

- **Element-wise** attention mask

- Longformer and Bigbird use **block-wise** attention mask (64 tokens as a **block**), for the sake of computation



- **Sparser pattern**

Length	Longformer	BigBird	Diffuser			
			tot	loc	glob	rand
1024	62.5	55.7	24.0	18.0	4.2	1.9
2048	34.4	32.5	15.5	9.2	4.2	2.1
4096	18.0	16.9	11.2	4.6	4.3	2.2

Diffuser — theoretical expressive power

- Diffuser as a **universal approximator**
 - Similar to standard Transformers, Diffuser can achieve universal **sequence-to-sequence** (or sequence-to-one) mapping
 - **Theorem 1** (informal): For any continuous function f , there exists a Diffuser model g satisfying $d(f, g) = \int ||f(X) - g(X)|| dX \leq \epsilon$, for any $\epsilon > 0$
 - Compared to other Sparse Transformers, Diffuser achieves the **same theoretical expressive power** with **fewer attention layers** under the same attention pattern.
 - Intuitively, this is because attention diffusion expands the receptive field to reach long-range tokens with the same layer

Diffuser — theoretical expressive power

- Diffuser as **expander graphs**
 - Expander graph: a **sparse** graph that has strong connectivity properties
 - **Theorem 2** (informal): The sparse pattern of Diffuser is a (d, ϵ) —**expander graph**, which can approximate the full-attention (complete graph), up to error ϵ .
 - **Theorem 3** (informal):
 - (1) Attention diffusion **mixes rapidly** on the sparse pattern of Diffuser
 - (2) Attention diffusion **improve the expander property** of the Diffuser sparse pattern, and thus improve its approximation power

Experiment setting

- **Datasets:**

- Text:

- MLM: Books, CCNews, Wikipedia
 - Classification: HYP, 20News Group, IMDB, Amazon
 - QA: WikiHop, TriviaQA

- Image density estimation: CIFAR10, TinyImagenet
 - Long range arena (LRA): a multi-domain dataset

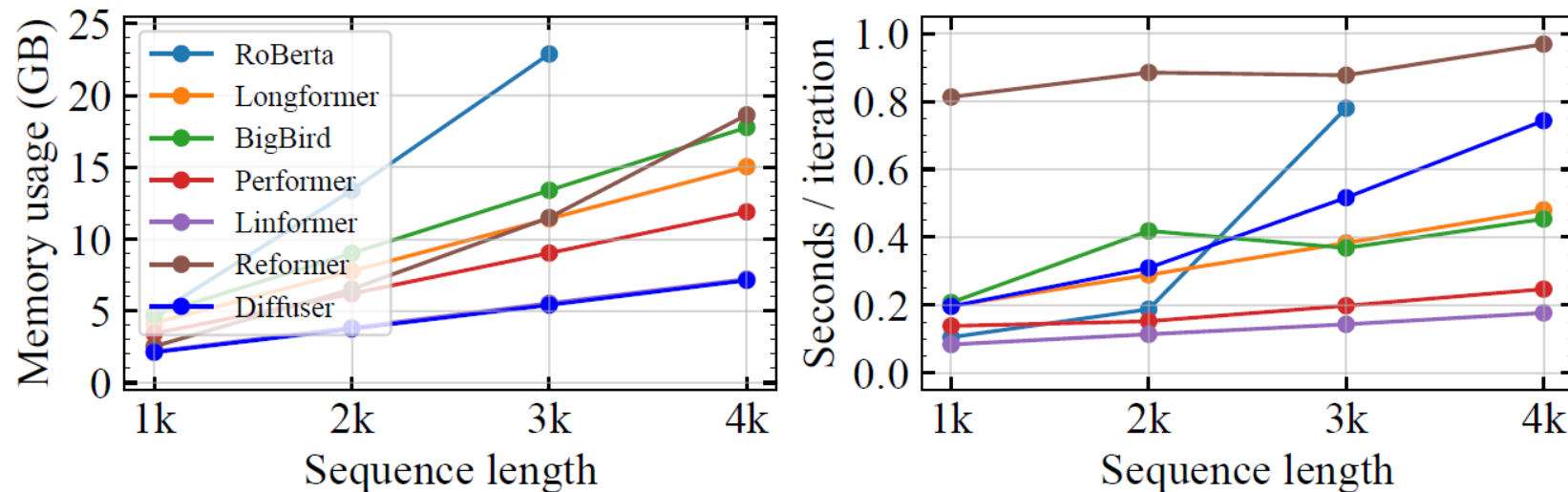
- **Implementation:** PyTorch + DGL

- **Model:** Bert-based for NLP task, and ViT-B-based model for CV.

- **Hardware:** Multi-GPU (RTX 3090) training with DDP

Experiments — Efficiency

- Diffuser is implemented with DGL (Deep Graph Library) packages for efficient sparse attention calculations (under message passing framework).



- Diffuser achieves **1.67× memory savings** in average of input lengths, with comparable running time compared to baselines.

Experiments — Performance

- Language **pretraining tasks**:
 - Diffuser is pretrained with masked language modeling (MLM) task, using Books, CC-News and Wikipedia

Model	BPC	Model	BPC
RoBERTa	2.02	Diffuser-init	3.52
Longformer	1.86	Diffuser-10K steps	1.96
BigBird	1.82	Diffuser-50K steps	1.68

- Diffuser achieves lower BPC (bits per character) compared to benchmarks after training for 50K steps.

Experiments — Performance

- Language **finetuning** tasks

Text classification

	HYP	20NG	IMDB	A-512	A-2048	Avg.
95pt.	2,030	1,229	771	1,696	5,216	-
BERT	85.7	85.3	91.3	59.2	50.3	74.36
RoBERTa	87.4	85.7	95.3	65.0	57.9	78.26
BigBird	92.2	82.3	95.2	67.4	<u>63.6</u>	80.14
Longformer	<u>93.8</u>	86.3	95.7	67.3	61.2	80.86
BigBird_D	93.1	84.5	95.0	68.2	63.4	80.84
Longformer_D	93.5	87.3	<u>95.4</u>	67.0	62.5	<u>81.24</u>
Diffuser	94.4	<u>86.8</u>	<u>95.2</u>	<u>67.8</u>	64.8	81.80

Question answering

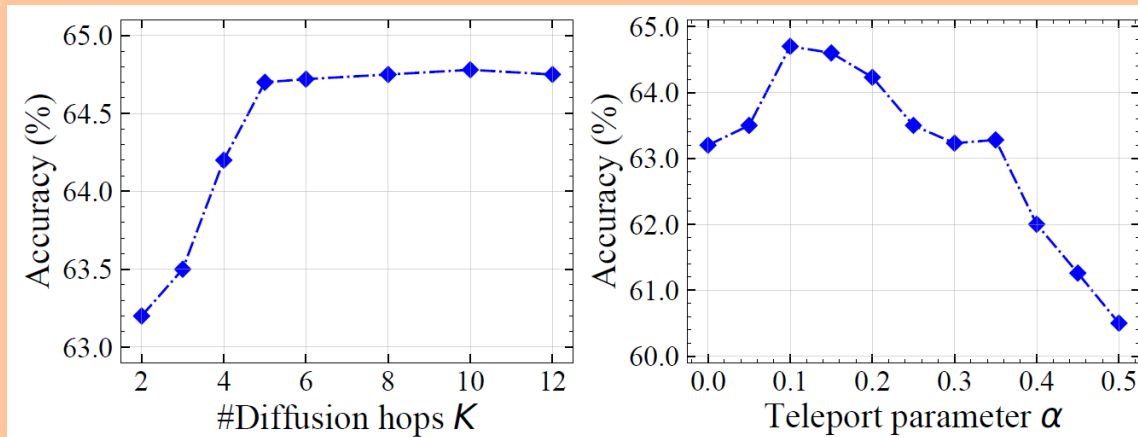
Model	WikiHop	TriviaQA	
Metric	Acc	F1	EM
RoBERTa	71.82	74.02	66.87
Longformer	75.30	74.82	67.24
BigBird	74.54	73.16	68.26
Diffuser	75.80	75.84	70.20

- Diffuser outperforms baselines on different downstream tasks, with better performance on **longer sequences** as in HYP, A(Amazon)-2048.
- Diffuser shows strong performance as well in **image generative** and **Long Range Arena (LRA)** benchmarks.

Experiments — Ablation Study

Model	ListOps	Text	Retrieval	Image	Pathfinder	Avg
#0 Diffuser	37.52	<u>65.20</u>	<u>61.28</u>	45.20	76.58	57.31
#1 w/o loc.	35.28	58.60	58.05	38.07	73.25	52.65
#2 w/o rand.	36.38	64.89	60.60	43.48	73.36	55.74
#3 w/o glob.	36.52	63.53	61.07	42.35	72.47	55.19
#4 w/o diff.	33.48	59.73	58.25	39.28	71.08	52.36
#5 w uni.	<u>37.39</u>	65.25	61.42	<u>44.98</u>	<u>76.30</u>	<u>57.07</u>

- **Local attention (#1)** and **attention diffusion (#4)** are important to the performance of Diffuser



- Performance is saturated with total diffusion step $K \geq 5$, and thus we use **$K = 5$**
- There exists optimal teleport parameter **$\alpha = 0.1$**

Conclusion

- We introduce attention diffusion for sparse transformers, targeting long-sequence task, and propose **Diffuser** model based on it.
- Diffuser achieves improved efficiency with sparser attention pattern.
- We show the expressivity of Diffuser, theoretically and experimentally.
- Diffuser achieves superior performance in language modeling, image modeling, and other long sequence modeling tasks

Summary of the Lecture

- **Efficiency** becomes more and more important as modern models scale
 - **Environmental cost** is a factor we should be aware of
- Key techniques to boost efficiency of deep learning models
- **Model compression**
 - **Distillation and finetuning.** There are additional side benefits (generalization)
- **Learning Techniques**
 - **Quantization** can be performed at inference (also enables edge device computing)
 - **Sparse** structure of weight and attention
 - **System and architecture research** is very needed to enable sparse-computation-friendly model development toolkit
- **Automation**
- **Infrastructure**